

Load Balancing and Data Locality in Adaptive Hierarchical N -Body Methods: Barnes–Hut, Fast Multipole, and Radiosity

JASWINDER PAL SINGH, CHRIS HOLT, TAKASHI TOTSUKA, ANOOP GUPTA, AND JOHN HENNESSY

Computer Systems Laboratory, Stanford University, Stanford, California 94305-4070

Hierarchical N -body methods, which are based on a fundamental insight into the nature of many physical processes, are increasingly being used to solve large-scale problems in a variety of scientific/engineering domains. Applications that use these methods are challenging to parallelize effectively, however, owing to their nonuniform, dynamically changing characteristics and their need for long-range communication. In this paper, we study the partitioning and scheduling techniques required to obtain effective parallel performance on applications that use a range of hierarchical N -body methods. To obtain representative coverage, we first examine applications that use the two best methods known for classical N -body problems: the Barnes–Hut method and the fast multipole method. Then, we examine a recent hierarchical method for radiosity calculations in computer graphics, which applies the hierarchical N -body approach to a problem with very different characteristics. We find that straightforward decomposition techniques which an automatic scheduler might implement do not scale well, because they are unable to simultaneously provide load balancing and data locality. However, all the applications yield very good parallel performance if appropriate partitioning and scheduling techniques are implemented by the programmer. For the applications that use the Barnes–Hut and fast multipole methods, simple yet effective partitioning techniques can be developed by exploiting some key insights into both the methods and the classical problems that they solve. Using a novel partitioning technique, even relatively small problems achieve 45-fold speedups on a 48-processor Stanford DASH machine (a cache-coherent, shared address space multiprocessor) and 118-fold speedups on a 128-processor simulated architecture. The very different characteristics of the radiosity application require a different partitioning/scheduling approach to be used for it; however, it too yields very good parallel performance. © 1995 Academic Press, Inc.

1. INTRODUCTION

Hierarchical N -body methods, which are based on a fundamental insight into the nature of many physical processes, are increasingly being used to solve a wide variety of scientific and engineering problems. Since these methods also afford substantial parallelism, applications that use them are likely to be important users of high-performance multiprocessors. Such applications, however, typically have characteristics that make it challenging to partition

and schedule them for effective parallel performance. In particular, the workload distribution and communication patterns across the logical units of parallelism (the n bodies, for example) are nonuniform and change as the computation proceeds, thus complicating the simultaneous incorporation of load balancing and data locality.

In this paper, we study partitioning and scheduling issues in representative applications that use three important hierarchical N -body methods. Two of these methods—the Barnes–Hut method [1] and Greengard and Rokhlin's fast multipole method (FMM) [15]—are the best methods known for classical N -body problems, such as those in astrophysics, electrostatics, and molecular dynamics. In addition to classical problems, the hierarchical N -body approach has recently been applied with great success to many different domains, including a problem with very different characteristics: radiosity calculation for global illumination in computer graphics [16]. This constitutes our third application, and we believe the three together provide adequate and wide-ranging coverage of the use of the hierarchical N -body approach today.

For each application, we first examine straightforward partitioning/scheduling techniques that a compiler or automatic scheduler might implement if it knew the appropriate parallelism. We find that these techniques do not provide scalable parallel performance—although some of them suffice for small-scale parallelism on the classical applications—since they are unable to simultaneously provide load balancing and the desired locality of data access. However, all the applications yield excellent and scalable parallel performance if more sophisticated techniques are implemented by the programmer.

The only other parallel version of a nonuniform hierarchical N -body application is a message-passing implementation of an astrophysical Barnes–Hut application [23]. It uses an orthogonal recursive bisection (*ORB*) partitioning technique to obtain both load balancing and data locality. (It is also substantially complicated by the lack of a shared-address-space programming model [28]). We propose a new partitioning technique called *costzones* for the classical applications. *Costzones* is much simpler to implement than *ORB* and performs better on shared-address-space machines particularly as the number of processors increases. We also extend both the *ORB* and *costzones* techniques to obtain good performance on our other classical applica-

tion, the FMM. We find that *costzones* is more easily extended and yields better performance than *ORB* on the FMM as well. Using *costzones*, we obtain 45-fold speedups on a 48-processor Stanford DASH machine, a cache-coherent shared address space multiprocessor, on both classical applications, and 118-fold speedups on a 128-processor, simulated shared address space architecture for the Barnes–Hut application, despite the relatively small problem sizes we run.

While the radiosity application uses the hierarchical N -body approach, the characteristics of this application are different enough from those of classical N -body applications that the *costzones* and *ORB* techniques are neither easily implementable nor useful in partitioning it. Although the classical applications require periodic repartitioning, explicit repartitioning by the user suffices in those cases and no on-the-fly stealing of tasks from other partitions is necessary for load balancing. The radiosity application, on the other hand, is far more unpredictable at runtime and requires dynamic task-stealing. Fortunately, there is substantial reuse of data within a task in this application, and simple techniques to preserve locality across tasks are successful in keeping the communication overhead low despite task-stealing. We obtain 27-fold speedups on a 32-processor DASH machine for the radiosity application.

Section 2 describes the insight behind hierarchical N -body methods and discusses our choice of applications and the coverage of hierarchical N -body methods that we achieve with them. In Section 3, we describe the applications we study and the algorithms they employ. Section 4 describes the multiprocessor environments in which we run our applications. In Section 5, we outline the goals of our parallel implementations and the common characteristics of the applications that complicate the achievement of these goals. Section 6 describes the metrics we use to compare different partitioning and scheduling techniques and outlines the organization of our experiments. Sections 7 to 9 discuss partitioning and scheduling techniques for the classical (Barnes–Hut and FMM) and radiosity applications in detail and present the results we obtain. Finally, Section 10 concludes the paper.

2. HIERARCHICAL N -BODY METHODS

Increasingly popular hierarchical algorithms in general are based on the following fundamental insight into the physics of many natural phenomena: Many physical systems exhibit a large range of scales in their information requirements, in both space and time. That is, a point in the physical domain requires progressively less information less frequently from parts of the domain that are further away from it. Hierarchical algorithms exploit the range of spatial scales to efficiently propagate global information through the domain. Prominent among these algorithms are N -body methods, multigrid methods, domain decomposition methods, multi-level preconditioners, adaptive mesh-refinement algorithms, and wavelet basis methods

[5]. Our focus in this paper is on hierarchical N -body methods.

The classical N -body problem models a physical domain as a system of n discrete bodies and studies the evolution of this system under the influences exerted on each body by the whole ensemble.¹ If long-range interactions cannot be ignored, the simplest and exact method is to directly compute all $O(n^2)$ pairwise interactions, which is prohibitive for large systems. Hierarchical tree-based methods have therefore been developed that reduce the complexity to $O(n \log n)$ or even $O(n)$ for uniform distributions [1, 15]² without losing much accuracy. The insight translates to the following in the N -body case: A group of particles may be approximated by a single equivalent particle if it is far enough away from the point at which its effect is being evaluated. The hierarchical application of this insight implies that the farther away the particles, the larger the group that can be thus approximated.

The same hierarchical methods are applicable to all the classical N -body problems, as well as to problems in fluid dynamics, integral equations, and numerical complex analysis. The partitioning and scheduling issues for these are also similar across domains. We therefore use a single application, the simulation of galaxies in astrophysics, as being representative of classical problems. While several hierarchical methods have been proposed for classical problems [1, 7, 15, 18, 29], the most widely used and promising among these are the Barnes–Hut method [1] and the fast multipole method [15], so we focus on these. In addition to classical problems, the hierarchical N -body approach has recently been used to efficiently compute radiosity in computer graphics. This application has characteristics very different from those of the classical problems. Including it as the third application in our study provides us with very good coverage of the use of the hierarchical N -body approach today.

3. THE PROBLEMS AND ALGORITHMS

Let us now examine the applications we study and the hierarchical methods they use. We first present the gravitational problem, and the Barnes–Hut and fast multipole methods that our applications use to solve it. Then, we describe the radiosity application. In every case, we discuss the available parallelism and the levels at which we exploit it.

3.1. The Classical N -Body Problem

Our example classical problem is a galactic simulation, which studies the evolution of a system of particles (a galaxy or set of galaxies) under the influence of Newtonian gravitational attraction. A particle is modeled as a point mass.

¹ We use the terms “body” and “particle” interchangeably in this paper.

² We show in [24] that the complexity of the adaptive FMM is not $O(n)$ as claimed in [14].

The time period for which the physical system's evolution is studied is discretized into hundreds or thousands of time-steps (Fig. 1). Every time-step involves several phases of computation, such as computing the forces acting on each particle and updating particle properties such as position and velocity. The phase which computes the interactions (forces) among particles is by far the most time-consuming in typical applications, and it is this phase that hierarchical methods speed up.

All the hierarchical methods for classical problems build a tree-structured, hierarchical representation of physical space and compute interactions by traversing the tree. This tree representing physical space is the main data structure in both the Barnes–Hut and fast multipole methods. The structure of the tree changes across time-steps, since the particle distribution and its bounding box change with time. The root of the tree represents this bounding box, i.e., a space cell containing all the particles in the system. The tree is built by recursively subdividing space cells until some termination condition, usually specified as the maximum number of particles allowed in a leaf cell, is met. In three dimensions, every subdivision of a cell results in the creation of eight equally sized children cells, leading to an octree representation of space; in two dimensions, a subdivision results in four children, leading to a quadtree. Let us now describe the methods and their use of the trees in some detail.

3.1.1. The Barnes–Hut Method

In the Barnes–Hut method, the force-computation phase within a time-step is expanded into three phases:

1. **Building the tree:** The current positions of the particles are first used to determine the dimensions of the root cell of the tree. The tree is then built by adding particles one by one into the initially empty root cell and subdividing a cell into its eight children as soon as it contains more than a single particle (the termination condition for the Barnes–Hut method). The result is a tree whose internal nodes are space cells and whose leaves are individual particles. Empty cells resulting from a cell subdivision are ignored. The tree (and the Barnes–Hut algorithm) is therefore adaptive in that it extends to more levels in regions

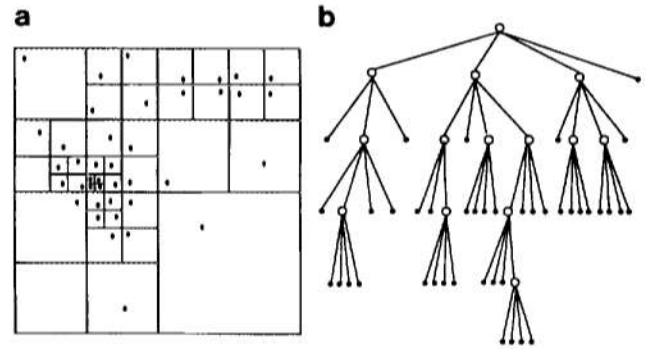


FIG. 2. A two-dimensional particle distribution and the corresponding quadtree. (a) Spatial domain; (b) quadtree representation.

that have high particle densities. Although we actually use a three-dimensional Barnes–Hut problem, Fig. 2 shows a small two-dimensional example domain and the corresponding quadtree. Since n particles are being loaded into an initially empty tree, and since the expected height of the tree when the i th particle is inserted is $\log i$, the expected computational complexity of this phase is $\sum_{i=1}^n (\log i) = n \log n$.

2. **Computing cell centers of mass:** Once the tree is built, the next step is to compute the centers of mass of all internal cells, since cells are approximated by their centers of mass when permitted by the hierarchical force-computation algorithm (see below). An upward pass is made through the tree, starting at the leaves, to compute the centers of mass of internal cells from the centers of mass of their children. The expected number of cells in the tree, and hence the expected computational complexity of this phase, is $O(n)$.

3. **Computing forces:** The tree is then used to compute the forces acting on all the particles. The force-computation phase consumes well over 90% of the sequential execution time in typical problems and is described in detail below.

Our parallel implementation also adds a separate partitioning phase to each time-step. Let us look at the force-computation algorithm in more detail.

The Force Computation Algorithm. In the Barnes–Hut method, the tree is traversed once per particle to compute the net force acting on that particle. The force-computation algorithm for a particle P starts at the root of the tree and conducts the following test recursively for every cell it visits: If the cell's center of mass is far enough away from P , the entire subtree under that cell is approximated by a single "particle" at the cell's center of mass, and the force that this center of mass exerts on the particle is computed; if, however, the center of mass is not far enough away from the particle, the cell must be "opened" and each of its subcells visited. A cell is determined to be far enough away if the condition $l/d < \theta$ is satisfied, where l is the length of a side of the cell, d is the distance of particle P

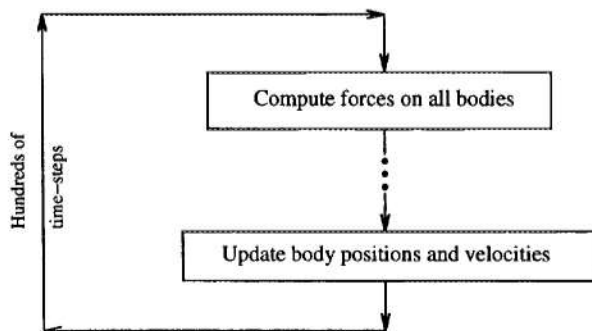


FIG. 1. Top-level flowchart of the classical N -body problem.

from the center of mass of the cell, and θ is a user-defined parameter used to control the accuracy of the computed forces. In this way, a particle P traverses more levels of those parts of the tree which represent space that is physically close to it and groups other particles at a hierarchy of length scales. The complexity of the force-computation phase typically scales as $(1/\theta^2) \log n$ for realistic values of θ [17].

The Available Parallelism. Each of the phases in a time-step can be executed internally in parallel. We do not exploit parallelism across phases or time-steps explicitly, except to avoid synchronizing between phases when possible.³ The natural unit of parallelism in all phases is a particle, except in computing the cell centers of mass, where it is a cell.

The tree-building and center of mass computation phases require both interprocessor communication and synchronization. In the tree-building phase, different processors may simultaneously try to modify the same part of the tree, so that cell-level mutual exclusion is required at the point of modification to preserve correctness. In the center of mass computation phase, a processor computing the center of mass of a cell C has to wait until the centers of the mass of all C 's children are computed, which requires cell-level event synchronization to preserve dependencies.

The force-computation phase requires communication but not synchronization. While the force computation for a particle requires position and mass information from other particles and cells, that information is not modified during the force-computation phase (positions are modified later in the update phase, and masses are not modified at all). Finally, the work for a particle in the update phase is entirely local to that particle.

3.1.2. The Fast Multipole Method

The FMM also uses a recursive decomposition of the computational domain into a tree structure, as well as a strategy of approximating cells of the tree by equivalent single "particles" when they are far enough away. In fact, both the FMM and the Barnes-Hut method have been shown to satisfy the same recursive set of equations, only using different elementary functions [19]. The primary differences between the two methods are the following:

- While the Barnes-Hut method directly computes only particle-particle or particle-cell interactions, the FMM also computes interactions between internal cells directly. It is this property that makes the force-computation phase in the FMM $O(n)$ rather than $O(n \log n)$ for uniform particle distributions.

³ It is possible to exploit fine-grained dependence information and replace global interphase synchronization by finer-grained (cell- or particle-level) synchronization. This is not likely to be useful for the problem and machine sizes we use, but may become more useful on much larger-scale machines.

- Whether a particle/cell is far enough away or "well-separated" from another cell is determined differently. In the FMM, whether two cells are well-separated from each other is determined entirely by their lengths and the distance between them (see Fig. 3), not by a user-adjustable parameter like the θ in the Barnes-Hut method.

- The FMM approximates a cell not by its center of mass but by a higher-order series expansion of particle properties about the geometric center of the cell. This series expansion is called the *multipole expansion* of the cell. The number of terms used in a multipole expansion determines the accuracy of the representation.

- The FMM uses analytic observations about its series expansions to establish tighter bounds on the errors in its approximations. The accuracy of force computation is controlled not by changing which cells are considered far enough away (as in Barnes-Hut), but by changing the number of terms used in the series expansions.

- The Barnes-Hut algorithm is inherently adaptive: It does not make any assumptions about the distribution of particles. The FMM, however, comes in two versions: a simpler version that is well-suited to uniform particle distributions, and a substantially more complex version that adapts arbitrary distributions. We use the adaptive FMM in this paper.

- While the mathematics of the Barnes-Hut algorithm is the same in three dimensions as it is in two, the FMM uses a different, more complex mathematical foundation in three dimensions [14]. The new mathematics substantially increases the constant factors in the time-complexity expression, so that the three-dimensional FMM is far more expensive than the two-dimensional FMM. The structures of the two- and three-dimensional algorithms are the same, however, as are the issues in parallelization. Since a sequential version of the three-dimensional adaptive FMM had not been implemented at the time of this work, we use the two-dimensional version.

- The FMM is far more complicated to program than the Barnes-Hut method.

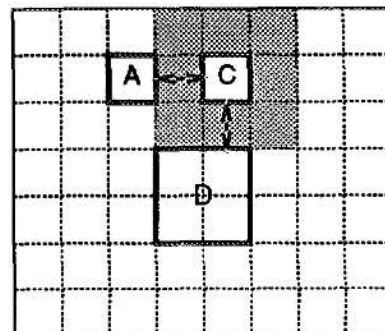


FIG. 3. Well-separatedness in the FMM. In the FMM, a cell is considered far enough away or "well-separated" from another cell b if its separation from b is greater than the length of b . Thus, cells A and C are well-separated from each other. Cell D is well-separated from cell C.

hence parallelism is a cell, not a particle. The tree-building and multipole expansion phases, as well as the downward pass in the force-computation phase, require both communication and synchronization. The list interactions, however, involve read-only communication and can be executed without synchronization. Finally, the update phase requires neither synchronization nor communication. Both the Barnes–Hut and FMM applications thus afford substantial parallelism, although the parallelism during the upward and (in FMM) downward passes through the tree is limited near the root.

3.2. The Radiosity Application

Computing the global illumination in a scene is an important problem in computer graphics. The two dominant approaches to solving this problem are the ray-tracing and radiosity methods. The radiosity method [8], which is view-independent and based on the physics of light transport, has been more successful in producing realistic images of complex scenes. It accounts for both direct illumination by light sources and indirect illumination through multiple reflections and is an essential tool in fields such as architectural design which require “photo-realistic” images.

The radiosity of a surface is defined as the light energy leaving the surface per unit area. Given a scene, the radiosity problem is to compute the radiosities of all surfaces, and hence the equilibrium distribution of light in the scene. In the traditional radiosity approach, the large polygonal surfaces that describe the scene (such as walls or desktops) are first subdivided into small enough elements that the radiosity of an element can be approximated as being uniform. The radiosity of an element i can be expressed as a linear combination of the radiosities of all other elements j . The coefficients in the linear combination are the “form factors” between the elements, where the form factor F_{ji} from element j to element i is the fraction of light energy leaving j which arrives at i . This leads to a linear system of equations, which can be solved for element radiosities. Computing the form factors between all pairs of n elements has $O(n^2)$ time complexity. However, the form factor decays with the inverse square of the distance between elements, which allows the use of an $O(n)$ hierarchical algorithm.

Although the hierarchical radiosity algorithm is based on the same basic approach as the algorithms we have seen for the classical *N*-body problem, it is very different in several ways. First, the starting point in the hierarchical radiosity algorithm is not the n finally undivided *elements* or “bodies,” but the much smaller number of large *input polygons* (walls, desktops) that initially represent the scene. These input polygons are hierarchically subdivided as the algorithm proceeds, generating a quadtree per polygon (see Fig. 6). In fact, the number of resulting leaf-level elements (n) in this forest of quadtrees is not even known *a priori*. We use the term *patch* for any intermediate pieces of surface in the quadtree, including the input polygons;

it is the equivalent of a “cell” or “internal cell” in the classical *N*-body applications.⁵

The second difference is that the interaction law in radiosity is not simply a function of the interacting entities and the distance between them. The interaction is the transport of light between patches, which can be occluded by other patches, making it necessary to compute the visibility between patches as part of determining form factors. Third, while the classical computations proceed over hundreds of time-steps, the radiosity computation proceeds over only a small number of iterations until the total radiosity of the scene converges. And finally, unlike the particles in the classical applications, the patches in radiosity do not move with time. Patches may, however, get further subdivided in successive iterations. These differences are what cause very different partitioning/scheduling techniques to be required for the radiosity application, as we shall see.

3.2.1. The Sequential Algorithm

The hierarchical radiosity algorithm [16] proceeds as follows. The input polygons that comprise the scene are first inserted into a binary space partitioning (BSP) tree [11] to facilitate efficient visibility computation between a pair of patches. Every input polygon is initially given a list of other input polygons which are potentially visible from it and with which it must therefore compute interactions. Polygon radiosities are then computed by the following iterative algorithm:

1. For every polygon, compute its radiosity due to all polygons on its interaction list, subdividing it or other polygons hierarchically as necessary (see Fig. 5, in which binary trees are shown instead of quadtrees for clarity).
2. Add all the area-weighted polygon radiosities together to obtain the total radiosity of the scene, and compare this with the total radiosity of the previous iteration to check for convergence within a fixed tolerance. If the radiosity has not converged, return to step 1. Otherwise, go to step 3.
3. Smooth the solution for display by computing the radiosities at the vertices of the leaf-level elements by interpolation. This phase is performed only once at the end of the application and is very simple from the viewpoint of parallelism.

Every input polygon can be viewed as the root of a quadtree that it will be decomposed into (see Fig. 5). Thus, the “tree” data structure used in this method is not a single tree that represents a spatial decomposition of the computational domain, but rather a forest of quadtrees representing individual polygons. The roots of these quadtrees (the input polygons themselves) are in turn the leaves of the BSP tree used for visibility testing. In every iteration, each of the (not yet fully subdivided) quadtrees is traversed

⁵ We sometimes use the term *patch* to include leaf-level elements in our description of the algorithm.

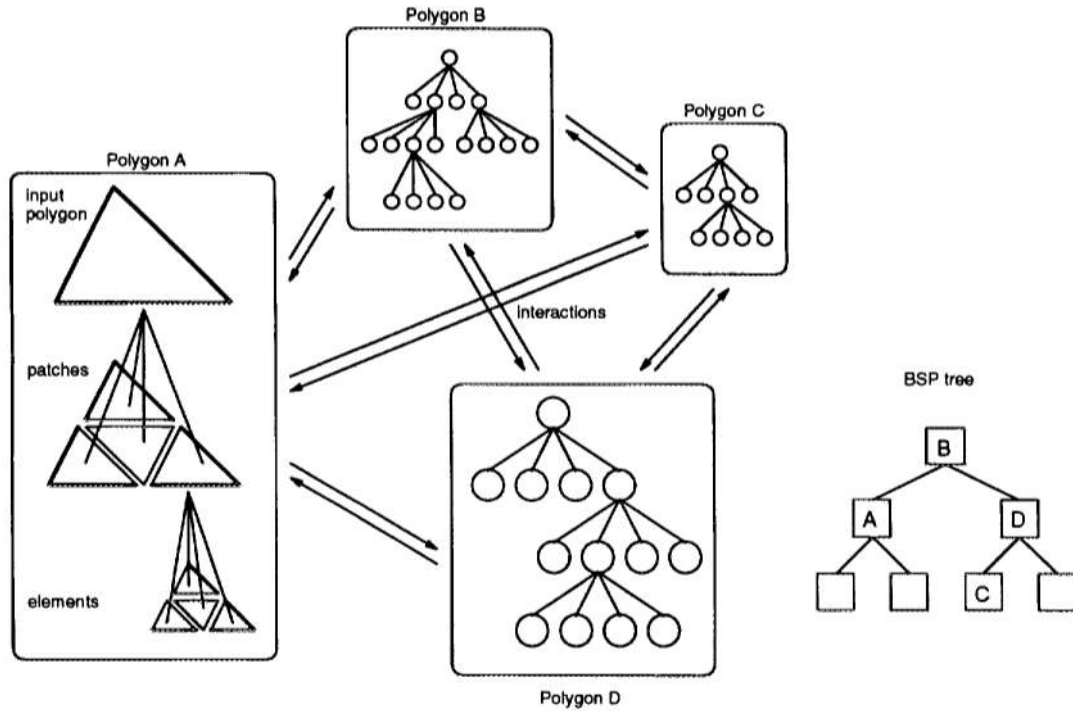


FIG. 5. High-level picture of polygons and interactions in the radiosity application.

depth-first starting from its root. At every quadtree node visited in this traversal, interactions of the patch (patch i , say) at that node are computed with all other patches, j , in its interaction list (see Fig. 6).⁶ The interaction between two patches involves computing both the visibility and the unoccluded form factor between them and multiplying the two to obtain the actual form factor (F_{ji}). The actual form factor (F_{ji}) multiplied by the radiosity (B_j) of patch j yields the magnitude of light transfer from j to i . Both the unoccluded form factor and the visibility are computed approximately, introducing an error in the computed (F_{ji}). An estimate (F_{err}) of this error is also computed. If the magnitude of error in the light-transfer ($B_j F_{err}$) is larger than a user-defined tolerance, the patch with the larger area is subdivided to compute a more accurate interaction. Children are created for the subdivided patch in its quadtree if they do not already exist.

If the patch being visited (patch i) is subdivided, patch j is removed from i 's interaction list and added to each of i 's children's lists. If patch j is subdivided, it is replaced by its children on patch i 's interaction list. Figure 6 shows an example of this hierarchical refinement of interactions. Patch i 's list is completely processed in this manner before its children are visited in the tree traversal. During this downward traversal, the radiosities gathered at the ancestors of patch i and at i itself are accumulated and passed to i 's children. Finally, after the traversal of a quadtree is completed, an upward pass is made through the tree to accumulate the area-weighted radiosities of every patch's

descendents into the radiosity of that patch. Thus, the radiosity of a patch is the sum of three quantities: the area-weighted radiosities of its descendants, the radiosity it gathers in its own interactions, and the radiosities gathered by its ancestors.

If the radiosity of the scene has not converged, the next iteration performs similar traversals of all quadtrees starting at their roots. At the beginning of an iteration, the interaction list of a patch in any quadtree is exactly as it was left at the end of the previous iteration, containing the patches with whom its interaction did not cause a subdivision.

3.3.3. The Available Parallelism

Parallelism is available at three levels in this application: across input polygons, across the patches that a polygon is subdivided into, and across the interactions computed for a patch. Since the quadtrees are built as the computation proceeds, all these levels of parallelism involve both communication and synchronization among processors. We will describe implementations that exploit different levels of parallelism in Section 9. The construction of the BSP tree is done sequentially, since the order in which patches are inserted into the BSP tree affects the total running time. Besides, construction of this tree takes negligible time, since only the original input polygons are inserted into it.

Before we present our results for the three applications, let us first describe the execution platforms that we use, the issues in effective parallelization on these platforms, and our experimental methodology.

⁶ A "patch" here could be an input polygon, an intermediate patch, or an element.

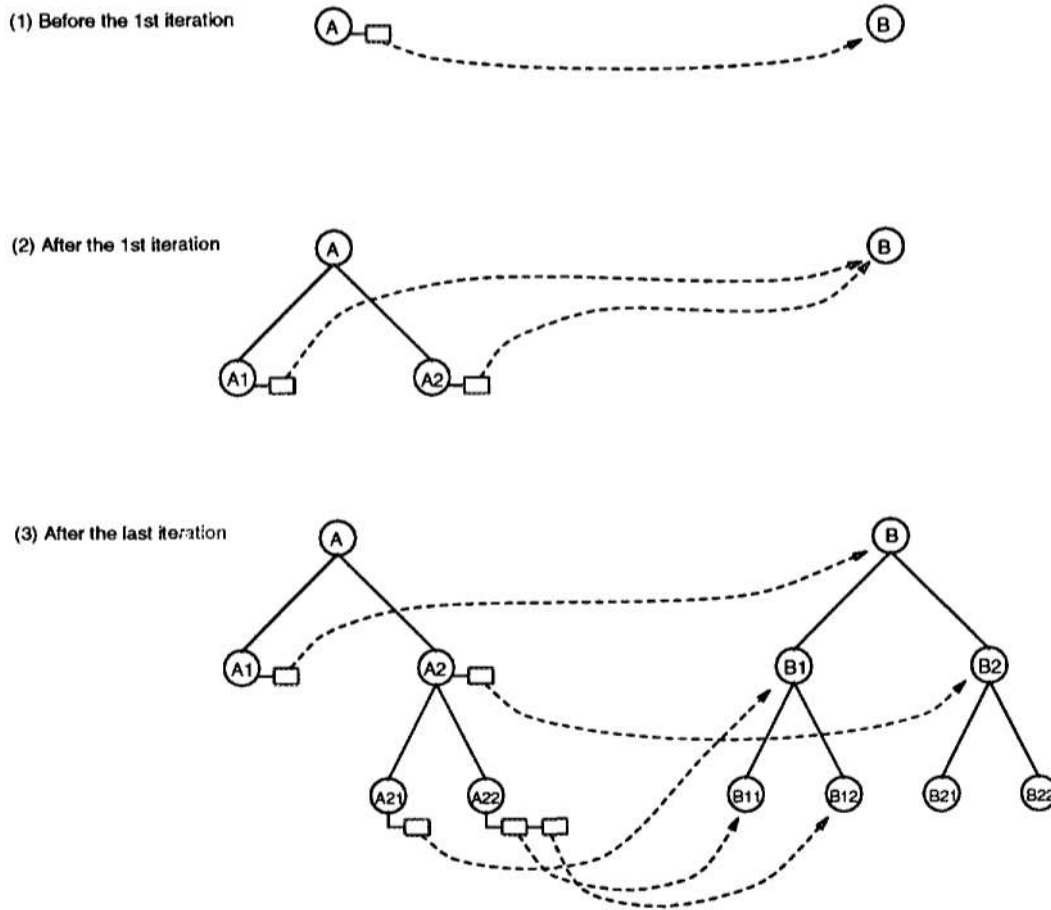


FIG. 6. Hierarchical interactions in the radiosity application.

4. THE EXECUTION PLATFORMS

Figure 7 shows the generalized shared address space multiprocessor that we assume in our parallelization. The multiprocessor consists of a number of processing nodes, connected by some general interconnection network. Every processing node comprises a processor, a cache, and an equal fraction of the total physical (main) memory on the machine. That is, the address space is shared but main memory is physically distributed. Caches are kept coherent by a hardware mechanism. We use two instantiations of this generalized multiprocessor in our experiments: the Stanford DASH Multiprocessor—a high-performance research machine—and a simulated multiprocessor.

The Stanford DASH Multiprocessor. The DASH machine [20] has 48 processors organized in 12 clusters.⁷ A cluster consists of four MIPS R3000 processors connected by a bus, and clusters are interconnected by a mesh network. Every processor has a 64 KB first-level cache and a 256 KB second-level cache, and every cluster has an equal

fraction of the physical memory on the machine. All caches in the system are kept coherent in hardware using a distributed directory-based protocol.

The Simulated Multiprocessor. While an existing machine such as DASH has the advantages of being fast and real, it is inflexible in configuration and unable to characterize program behavior of interest to us (inherent communication in the program, for example). To overcome these limitations, we also perform experiments on an event-driven simulator of an idealized shared-address-

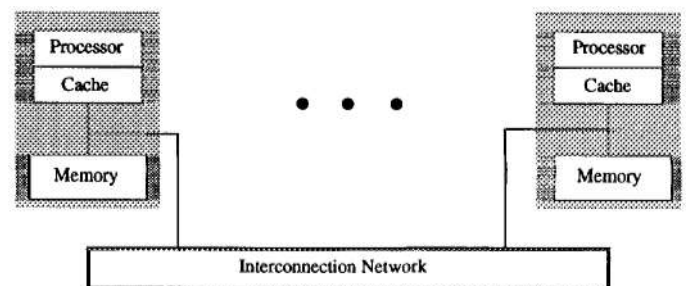


FIG. 7. The simulated multiprocessor architecture.

⁷ The prototype actually has 64 processors in 16 clusters, but is broken up into two separate machines in usual operation.

space multiprocessor [13]. The simulated multiprocessor looks exactly like that described in Fig. 7, with the simple, three-level, nonuniform memory hierarchy. The timing of a simulated processor's instruction set is designed to approximate that of the MIPS R3000 CPU and R3010 floating point unit. Access latencies in the memory system are assumed to be as follows. Hits in the issuing processor's cache cost a single processor cycle. Read misses that are satisfied in the local memory unit stall the processor for 15 cycles, while those that are satisfied in a remote cluster (cache or memory unit) stall the processor for 60 cycles. Since write miss latencies can be quite easily hidden by software/hardware techniques, the corresponding numbers for write misses are 1 and 3 cycles, respectively. By using constant miss latencies, we ignore contention in various parts of the system. However, the constant latencies keep our machine model simple and are appropriate for the characteristics we want to study.

5. ISSUES IN EFFECTIVE PARALLELIZATION

There are six sources of overhead that inhibit a parallel application from achieving good speedups over the best sequential implementation: inherently serial sections, redundant work, parallelism management, synchronization, load imbalance, and communication. As in many scientific applications, the first four are not very significant here for moderate-scale machines. Let us therefore define the scope of our treatment of the last two and the goals of our parallel implementations.

Load Balancing. The goal in load balancing is intuitive: Tasks in the workload should be assigned to processors in a way that minimizes the time any processor spends waiting for other processors to reach synchronization points.⁸

Data Locality. As we have seen, modern multiprocessors are built with hierarchical memory systems, in which processors have faster access to data that are closer to them in the hierarchy. Exploiting locality of data reference can increase the fraction of memory references that are satisfied close to the issuing processor, and hence improve performance. Many of the aspects of locality in a multiprocessor are no different from those in uniprocessor, since the part of the memory system that is within a processing node is very similar to a uniprocessor memory hierarchy. The key difference in multiprocessors is that they extend the hierarchy by introducing communication among processing nodes, which is very expensive. For this reason, we focus our discussion of locality primarily on reducing communication by scheduling tasks that access similar data in the same processor. By doing this, we ignore several more detailed locality issues⁹ as well as the impact of local-

ity in a network topology.¹⁰ We also do not address the impact of allocating data appropriately in physically distributed main memory in this paper, primarily because this is very difficult to do for these applications and because experiments we have performed show that it does not help performance very significantly, that the important form of locality is reuse or temporal locality [24]. Despite our focus on communication, we do make all reasonable efforts to exploit locality within a node effectively.

Since there is often a tension between load balancing and data locality, effective partitioning and scheduling techniques must strive to find a good compromise between these goals. Unfortunately, several characteristics of hierarchical N -body applications make it challenging to attain these goals simultaneously:

- The physical domain is typically highly *nonuniform*, which has implications for load balancing and communication:
 - The work done per unit of parallelism (particle or cell) is not uniform and depends on the distribution of particles in space. Equally sized spatial partitions or those with equal numbers of particles therefore do not have equal work associated with them.¹¹
 - The natural communication is fine-grained, long-range, and irregular despite its hierarchical nature. The amount of communication is nonuniformly distributed across particles/cells and hence across processors.
 - In the classical applications, the amount of communication falls off with distance equally in all directions. To reduce communication frequency and volume, therefore, a processor's partition should be spatially contiguous and not biased in size toward any one direction.
 - The *dynamic nature* of the applications—together with the nonuniformity—causes the distribution of particles and hence the work and communication distributions to change slowly across time-steps. There is no steady state. This means that static partitioning schemes are unlikely to work well.
 - The different phases in a time-step have different relative amounts of work associated with particles/cells, and hence different preferred partitions if viewed in isolation.

6. EXPERIMENTAL METHODOLOGY

Let us now describe the metrics we use to compare partitioning techniques and the organization of our experiments.

¹⁰ Locality in the network topology is not a significant issue in real machines until the number of processing nodes becomes very large, since the time for a packet to get in and out of the network dominates the time due to the number of network hops traversed. Also, the impact of network locality is diluted if other forms of locality (in the cache or local memory) are exploited effectively, and network locality is in any case orthogonal to these other forms of locality.

¹¹ In the problems we have run, the standard deviation in the distribution of work across units of parallelism (particles, say) was usually several times the mean work per unit.

⁸ A task in this paper refers to a unit of work which is treated as being indivisible from the scheduling point of view.

⁹ For example, prefetching data with long cache lines or in software, false-sharing of data, and cache mapping collisions.

6.1. Comparison Metrics

The primary metric we use for comparing different implementations is overall performance, i.e., the speedup they yield over the same sequential program. We present speedups obtained on the DASH multiprocessor in all cases and also those obtained on the simulator in cases where we could simulate large enough problems that speedups are meaningful. In addition to speedups, we also present results that separately compare the load balancing and communication behavior of different schemes. These results are obtained on the simulator, and we have corroborated their trends with the MTOOL [12] performance debugger on DASH.

We compare load balancing behavior by measuring the time that processes spend waiting at synchronization points. In comparing communication behavior, we focus on inherent communication in the program that implements a given scheme, rather than on how these inherent program characteristics interact with the details of a particular memory system configuration. For this reason, we simulate infinite per-processor caches with a small cache line size (8 bytes).

In general, cache misses in a multiprocessor arise from four sources: (i) *communication*, both inherent and due to false sharing of cache or memory blocks; (ii) replacements of useful data due to limited cache *capacity*; (iii) replacements due to cache mapping *conflicts* caused by an associativity smaller than the cache size; and (iv) *cold-start* effects. By using infinite caches, we eliminate capacity and conflict misses. The use of a small cache line virtually eliminates false-sharing misses. Cold-start misses are not significant in these applications in practice, since the applications are typically run for hundreds of time-steps. Since our experiments run the applications for only a very small number of time-steps, we explicitly ignore the cold-start period by resetting the simulator statistics after two time-steps (by which point the program has settled down and the partitioning scheme has already taken effect). Thus, our simulation environment essentially eliminates all misses other than those due to inherent communication. Our metric for comparing inherent communication in this environment is the average number of cache misses per 1000 processor busy cycles.

One implication of our use of infinite caches is that performance results obtained with the simulator may be biased in favor of partitioning schemes that have poor locality characteristics. The reason for this is that schemes with poor locality may have larger active working sets than schemes with good locality. On a real machine, schemes with good locality may therefore have an important performance advantage if the finite caches on the machine are large enough to accommodate the working sets of these schemes but not the working sets of schemes with poor locality. Infinite caches do not capture this effect. However, this is not a very significant issue in our applications since the important working sets are small [22]. Besides, infinite

caches are better at measuring inherent communication, which is what we want to compare using the simulator.

6.2. Organization of Experiments

For each application, we first examine approaches that are conceptually obvious and very easy for a programmer to implement. These approaches use only information that can be obtained statically from the program to guide their decisions about load balancing and data locality. They do not invoke any special mechanisms to account for characteristics of the input domain and are the techniques that a compiler or automatic scheduler would use if it knew the appropriate parallelism. After understanding the limitations of these techniques for an application, we move on to study the effectiveness of more sophisticated partitioning techniques. Let us now examine the individual applications.

7. THE BARNES-HUT APPLICATION

7.1. The Straightforward Approaches

As described in subsection 3.1.1, the sequential Barnes-Hut application is structured in phases, every phase consisting of an outer loop that iterates over particles (or cells, in one phase). Each iteration of these loops performs the work for a particle or cell in that phase, which may involve computing the forces on a particle or inserting a particle into the tree. Two easy partitioning techniques therefore suggest themselves. One is a fully *static* approach, in which every processor is assigned an equal set of iterations in every phase, the same iterations in every phase and time-step as far as possible. The other is a fully dynamic approach, in which processors dynamically obtain loop iterations or groups of iterations until there are none left in the loop. We call this second scheme *loadbalance*. Both of these are schemes that a compiler or automatic scheduler might implement.

Static approaches are appropriate for many scientific applications, in which the work per iteration in parallel loops is uniform and locality in iteration space translates naturally to the desired locality in the problem domain [26].¹² In our hierarchical *N*-body applications, however, the *static* scheme described above neither guarantees load balancing nor provides data locality. The load imbalance results from the fact that an equal number of particles does not imply an equal amount of work. However, the imbalance may not be severe in this scheme if the number of particles per partition is large. In fact, the *static* scheme is likely to compromise data locality more than load balancing for realistic problems. Although the scheme provides locality in the loop iteration space—as well as a form of

¹² This is true of some uniform *N*-body applications as well, particularly those that use a fixed cutoff radius in computing interactions and a spatial directory to efficiently find the particles that are within the cutoff radius [27].

object locality by always assigning a processor the same particles—the numbering of particles/cells in the loop iteration space has nothing to do with their positions in space and hence with the *physical locality* that is most important to performance.

The *loadbalance* scheme does not have a load imbalance, but it does not attempt to incorporate any form of data locality. Let us now examine some more sophisticated partitioning techniques. These techniques attempt to simultaneously provide load balancing and physical locality, by taking advantage of some insights into the application and the Barnes–Hut method. We first discuss how they incorporate load balancing and then physical locality.

7.2. Load Balancing

Whatever the technique used to provide locality, the relative amounts of work associated with different particles must be known if load balancing is to be incorporated without resorting to dynamic task stealing (which has overheads and compromises data locality). There are three problems in load balancing the Barnes–Hut application. First, the relative distribution of work among particles is not the same across the phases of computation (force calculation, update, etc.). Different phases might therefore have different preferred partitions. We focus our partitioning efforts on the force calculation phase, since it consumes well over 90% of the sequential execution time. Partitions that are appropriate for force calculation are also likely to be appropriate for the tree-building and center-of-mass computation phases, since all these phases associate more work with particles/cells in denser parts of the domain and have the same locality characteristics. The update phase will not be well load balanced by these partitions since it calls for an equal number of particles per processor. However, the fraction of the execution time spent in this phase is too small to make repartitioning worthwhile, at least for our problem and machine sizes.

The second problem is that the distribution of work among particles is not uniform even within the force-computation phase (see Section 5). The solution to this problem is to associate a cost with every particle, which reflects the amount of work needed to compute the force on that particle, and use this cost rather than the number of particles as the load balancing metric.

The third problem is that this cost of a particle is not known a priori and changes across time-steps. However, there is a key insight into the application characteristics that allows us to solve this problem. Since classical N -body problems typically simulate physical systems that evolve slowly with time, *the distribution of particles changes very slowly across consecutive time-steps*, even though the change in distribution from the first time-step to the last is often dramatic. In fact, large changes from one time-step to the next violate fundamental assumptions made in the simulation and imply that the time-step integrator being used is not accurate enough. Since a particle's cost

depends on the distribution of particles, the slow change in distribution implies that a particle's cost in one time-step is a good estimate of its cost in the next time-step.

In the Barnes–Hut application, a good measure of a particle's cost is simply the number of interactions (with other particles or cells) required to compute the force on that particle. This measure of cost is used independently by both Salmon [23] and us. The overhead of counting interactions is negligible: A process simply increments a per-particle counter every time it computes an interaction. As we shall see, this work-counting technique is very effective. In fact, it is a powerful technique for load balancing in many nonuniform problems that simulate slowly evolving, natural physical phenomena.

7.3. Providing Physical Locality

Partitioning the domain space statically for physical locality is clearly not good enough, since it will lead to very poor load balancing. In this subsection, we describe two partitioning techniques, *orthogonal recursive bisection* and *costzones*, that provide load balancing as well as physical locality. Both techniques use the work-counting mechanism of the previous subsection for load balancing, but they differ in the approach they take to providing physical locality.

7.3.1. Partitioning Space: Orthogonal Recursive Bisection

Orthogonal Recursive Bisection is a technique for providing physical locality in a problem domain by explicitly partitioning the domain space [10]. It was first used for hierarchical N -body problems in Salmon's message-passing Barnes–Hut implementation [23]. The idea in ORB partitioning is to recursively divide the computational domain space into two subspaces with equal costs, until there is one subspace per processor (see Fig. 8).

The cost of a subspace is defined as the sum of the profiled costs of all particles in the subspace. Initially, all processors are associated with the entire domain space. Every time a space is divided, the processors associated with the space are divided equally into two processor subsets, and each subset is assigned to one of the resulting subspaces. The Cartesian direction in which division takes place is usually alternated with successive divisions, although this alternation is sometimes violated to avoid resulting in long and thin subspaces. A parallel median finder is used to determine where to split the current subspace in the direction chosen for the split. The worst-case load imbalance per bisection can be made as small as the maximum amount of work associated with a single particle (there is a trade-off between this imbalance and the time spent in the median finder), but the imbalance in the final partitions might be somewhat worse owing to the cumulative effects of successive bisections. A more detailed description of an ORB implementation for the Barnes–Hut problem can be found in [23].

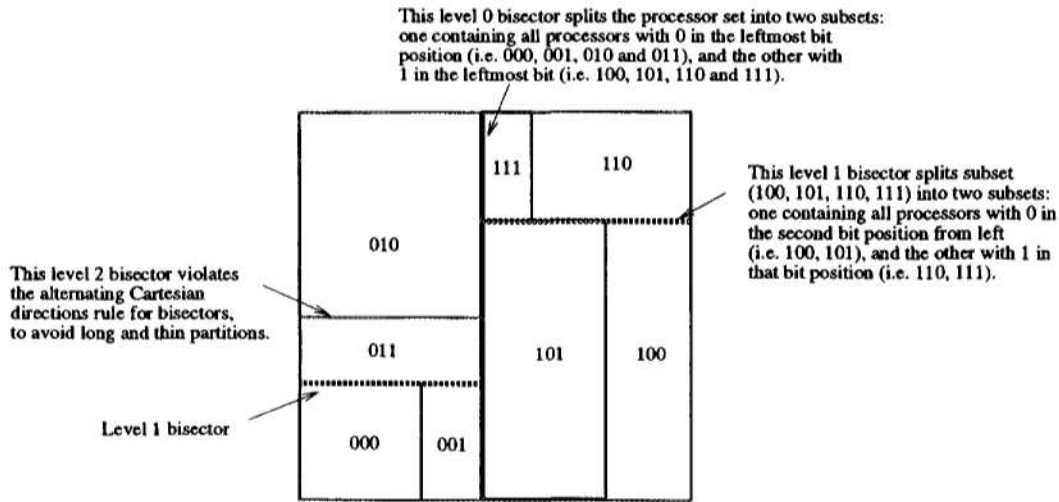


FIG. 8. ORB partitioning.

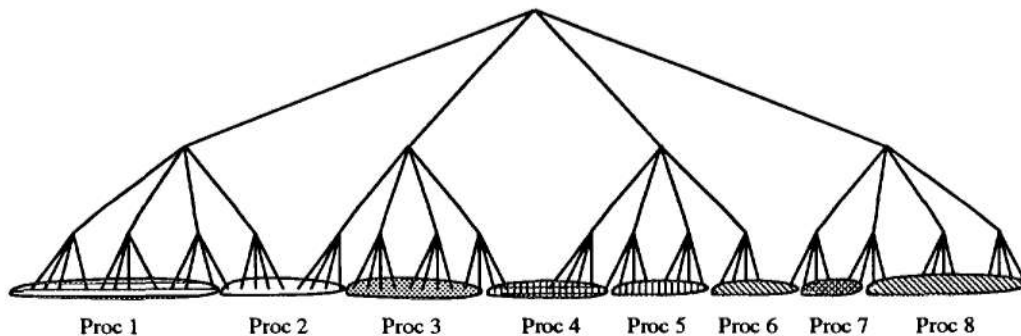
ORB introduces several new data structures, including a binary ORB tree that is distinct from the Barnes–Hut tree. The nodes of the ORB tree are the recursively subdivided subspaces, with their processor subsets, and the leaves are the final spatial partitions. ORB is nontrivial to implement and debug, and can have significant runtime overhead particularly as the number of processors increases (as we shall see). It also restricts the number of processors used to a power of two, although variants can be developed that do not have this restriction. For these reasons, we develop a simpler partitioning technique that is more flexible and scales better with the number of processors used.

7.3.2. Partitioning the Tree: Costzones

Our *costzones* partitioning technique takes advantage of another key insight into the hierarchical methods for classical *N*-body problems, which is that *they already have a representation of the spatial distribution encoded in the tree data structure they use*. We can therefore partition the tree rather than partition space. In *costzones*, the tree is conceptually laid out in a two-dimensional plane, with a cell's children laid out from left to right in increasing order

of child number. Figure 9 shows an example using a quad-tree. The cost of every particle, as counted in the previous time-step, is stored with the particle. Every internal cell holds the sum of the costs of all particles that are contained within it, these cell costs having been computed during the upward pass through the tree that computes the cell centers of mass.

The total cost in the domain is divided among processors so that every processor has a contiguous, equal range or zone of costs (hence the name *costzones*). For example, a total cost of 1000 would be split among 10 processors so that the zone comprising costs 1–100 is assigned to the first processor, zone 101–200 to the second, and so on. Which cost zone a particle belongs to is conceptually determined by the total cost up to that particle in an inorder traversal of the tree. In the *costzones* algorithm, processors descend the tree in parallel, picking up the particles that belong in their cost zone (most of the time, a processor picks up entire cells rather than individual particles). A processor therefore performs only a partial traversal of the tree. To preserve locality of access to internal cells of the tree in later phases, internal cells are assigned to the processors that own most of their children. A detailed description of the *costzones* algorithm can be found in [24]. The algo-

FIG. 9. Tree partitioning in the *costzones* scheme.

rithm introduces no new data structures, requires only a few lines of code, and has negligible runtime overhead (see the results in subsection 7.4).

The *costzones* technique yields partitions that are contiguous in the tree as laid out in a plane. How well this contiguity in the tree corresponds to contiguity in physical space depends on how the locations of cells in the tree map to their locations in space. This in turn depends on the order in which the children of cells are numbered (when conceptually laying them out from left to right in the planarized tree). The simplest ordering scheme, which is also the most efficient for determining which child of a given cell a particle falls into, is to use the same ordering for the children of every cell. Unfortunately, there is no single ordering which guarantees that contiguity in the planarized tree will always correspond to contiguity in space.

The partition assigned to processor 3 in Fig. 10 illustrates the lack of robustness in physical locality resulting from one such uniform ordering in two dimensions (children of all cells are numbered clockwise, starting from the bottom left child). While all particles within an internal cell or subtree are indeed in the same cubical region of space, particles (or subtrees) that are next to each other in the planarized tree may not have a common ancestor until

much higher up in the tree and may therefore not be near each other in physical space.

There is, however, a simple solution that makes contiguity in the planarized tree always correspond to contiguity in space. In this solution, the order in which children are numbered is not the same for all cells. However, the ordering of a cell *C*'s children is still easy to determine, since it depends on only two things: (i) the ordering of *C*'s parent's children (*C*'s siblings) and (ii) which child of its parent *C* is in that ordering.

Let us consider a two-dimensional example. Since every cell has four children in two dimensions, there are eight ways in which a set of siblings can be ordered: four possible starting points, and two possible directions (clockwise and counterclockwise) from each starting point. It turns out that only four of the eight orderings need actually be used. Figure 11a shows the four orderings we use in our example and illustrates how the ordering for a child is determined by the ordering for its parent. The arrow in a cell represents the ordering of that cell's children. For example, if the children are numbered 0, 1, 2, 3 in a counterclockwise fashion starting from the upper right, then in case (1) the children of the top-level cell are ordered 2, 1, 0, 3, and in case (2) the children are ordered 2, 3, 0, 1. The ordering of the children's children are also shown in Fig. 11a.

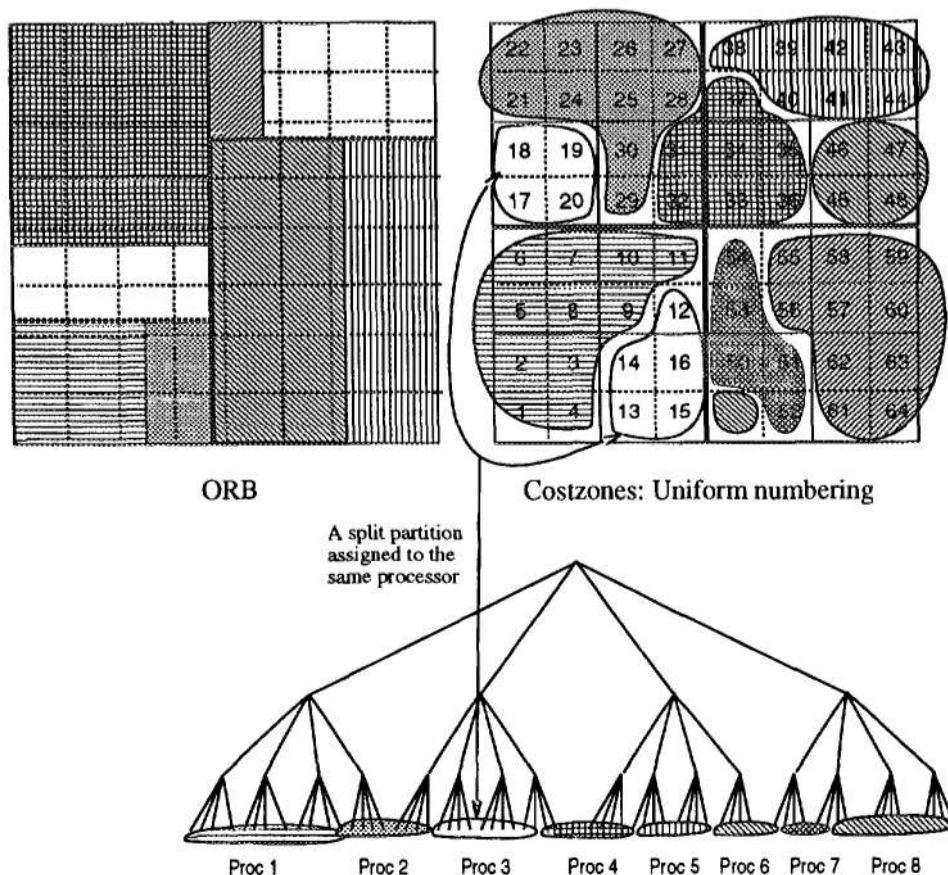


FIG. 10. ORB and *costzones* with uniform numbering.

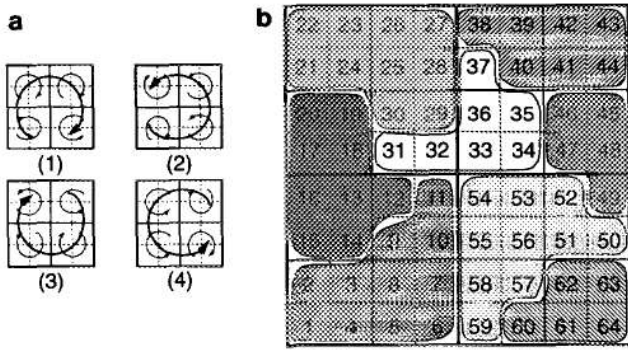


FIG. 11. *Costzones* with nonuniform numbering. (a) Ordering of cells; (b) resulting partitions.

Figure 11b shows the resulting partitions given the same distribution as in Fig. 10. All the partitions are physically contiguous in this case. The three-dimensional problem is handled similarly, except that there are now 32 different orderings used instead of 4. Details of the three-dimensional case can be found in [24].

Finally, although the simpler, uniform ordering scheme discussed earlier can result in noncontiguous partitions, this is not much of a performance issue on moderately sized shared address space machines with caches. Since the number of processors is typically small compared to the number of particles, a processor usually picks up large subtrees during *costzones* partitioning. The number of transitions between contiguous regions (subtrees) during force computation is small compared to the size of each region, and enough locality is obtained within a region that the transitions are not a problem. In our performance results in Section 7.4, we use the more robust, nonuniform child ordering in the partitioning scheme we call *costzones*.

Before we measure the performance of all the schemes we have described (*static*, *loadbalance*, *ORB*, and *costzones*), let us introduce a final scheme that helps us separate the effect of data locality from that of load balancing. This scheme, which we call *spacelocality*, uses the same

tree traversal technique for providing physical locality as *costzones*, but uses a naive metric—unit cost for every particle—for load balancing.

7.4. Results

We now present performance results for the Barnes–Hut application. The input distribution that we use consists of two Plummer model [1] galaxies, offset from each other in each of the three Cartesian directions. The Plummer model is an empirical model of galactic clusters. The density of a cluster is very large near the center and then diminishes with distance from the center. We have experimented with other realistic distributions as well, with similar results.

7.5. Speedups

Figure 12 shows speedups with 32 K particles on the DASH multiprocessor and with 8 K particles on the simulator with infinite caches. The accuracy parameter θ (see subsection 3.1.1) is set to 1.0. Measurements with other reasonable numbers of particles and accuracies show similar relative trends across partitioning schemes. All speedups are relative to a uniprocessor execution of the *static* scheme, which is almost identical to a sequential program execution. (48-processor speedups are not shown for ORB since 48 is not an integer power of 2.)

At least up to 16 processors, all schemes yield good speedups for both problem sizes, although the differences between them are clear. The differences grow as more processors are used. To facilitate comparing the different schemes, let us look separately at their load balancing and communication behavior, using results obtained from the simulator.

7.6. Load Balancing and Communication

For load balancing, Fig. 13a shows the amount of time processors spend waiting at synchronization events (locks

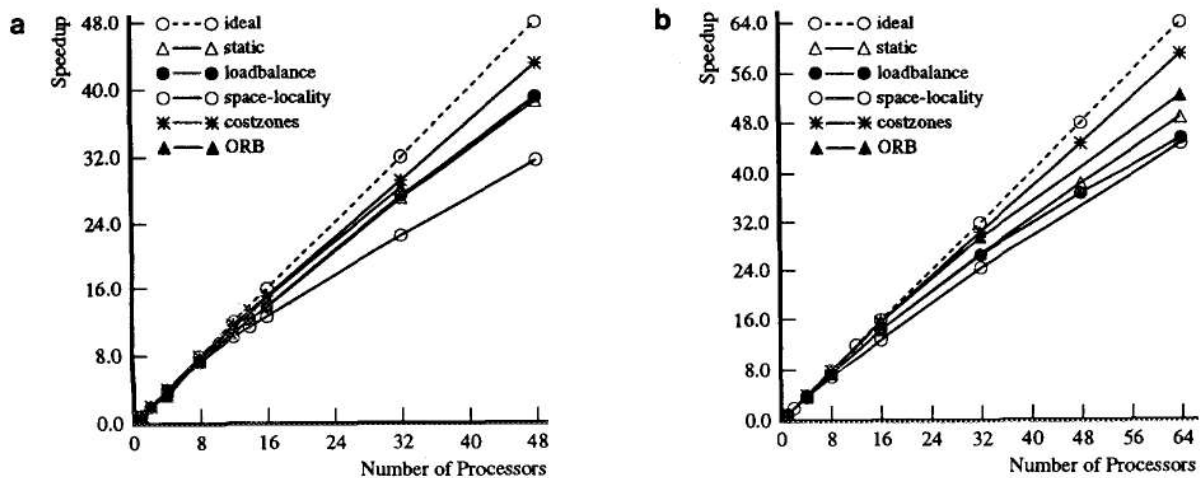


FIG. 12. Speedups for the Barnes–Hut application. (a) 32 K particles on DASH; (b) 8 K particles on simulator.

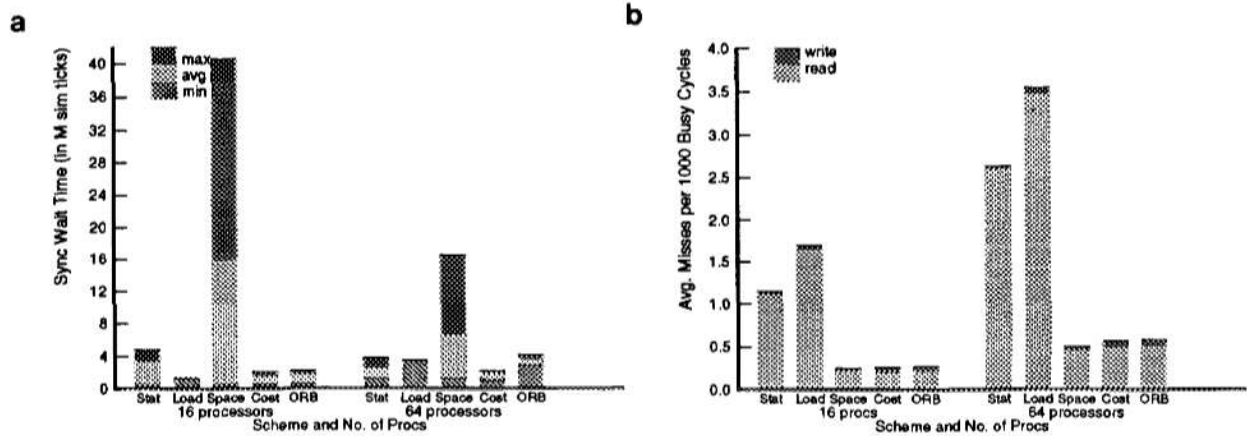


FIG. 13. Barnes-Hut behavioral statistics on the simulator ($n = 8\text{ K}$, $\theta = 1.0$). (a) Synchronization wait times; (b) communication.

and barriers) under the different schemes. The minimum, average and maximum time (in millions of simulated processor clock ticks) over all processors are shown for 16 and 64 processor executions. The total simulated execution time for the best scheme (*costzones*) was 68 million ticks with 16 processors, and 18 million ticks with 64 processors.

For communication, Fig. 13b shows the average number of misses in the infinite cache per thousand busy cycles per processor. Let us now relate the information in these figures to the performance of the different schemes in Fig. 12.

The Static and Loadbalance Schemes. Figures 13a and 13b verify our expectations about the *static* and *loadbalance* schemes that a compiler or automatic scheduler might implement. For example, Fig. 13a shows that the load balancing behavior of the *static* scheme is not bad since the workload characteristics average out over the large number of particles in every partition (of course, this is not guaranteed for all distributions). And Fig. 13b shows that although the communication behavior of the *static* scheme is better than that of *loadbalance*, the absence of physical locality makes this communication behavior quite poor relative to that of the best schemes.

It is interesting that even for the relatively small problem sizes used here, the *static* and *loadbalance* schemes are able to obtain quite good performance with small numbers of processors, despite their shortcomings in incorporating locality. This is because the amount of communication relative to computation is small for these ratios of problem size to number of processors. For example, although the *loadbalance* scheme has a communication to computation ratio that is 7 times that of the *costzones* scheme with 64 processors in Fig. 13b, its ratio is still less than 4 misses per 1000 processor busy cycles.

There are two reasons for the low communication to computation ratio. First, although the amount of private (not shared) data in the application is small, most of the references in the application are actually to private data. For every piece of shared data that a processor reads, it

performs a reasonably large amount of computation (a particle-cell interaction, say) in which it primarily references temporary, private data structures that are reused in other computations. For example, for an 8 K particle Barnes-Hut simulation with $\theta = 1.0$, only 15% of the references (23% of reads and 0.7% of writes) are to shared data. Second, although the *static* and *loadbalance* schemes cause every processor to reference data from almost the entire tree during force computation, many of these data—particularly at higher levels of the tree—are still reused substantially by the processor.¹³ The impact of communication and hence physical locality is likely to be much more significant when larger machines are used, however, as indicated by Fig. 12. This is particularly true since the number of particles is not expected to scale linearly with the number of processors [24, 27].

Besides performance benefits on large machines, physically contiguous partitions have other important advantages. They allow the use of a more efficient tree-building algorithm (see subsection 7.6.1) which helps alleviate an important performance bottleneck for large numbers of processors. They also reduce the working set size for force computation and have important implications for programming complexity and the sizes of problems that can be run on message-passing machines [24, 28].

The Spacelocality Scheme. Incorporating physical locality indeed leads to dramatically better communication behavior in the *spacelocality* scheme (see Fig. 13b). However, since a processor's particles are physically clumped together, giving every processor an equal number of parti-

¹³ The speedups of the *static* and *loadbalance* schemes are even closer to those of *costzones* on DASH than on the simulator, because of the effect of cache-to-cache sharing on the bus within a 4-processor cluster. Instead of every processor incurring a very high-latency read miss on every datum that has to come in from outside the cluster, only one processor in the cluster incurs this remote miss latency—other processors obtain the data from that processor's cache with a much lower latency. This additional, cluster level of the memory hierarchy makes the impact of read-sharing communication smaller on DASH than on the simulator.

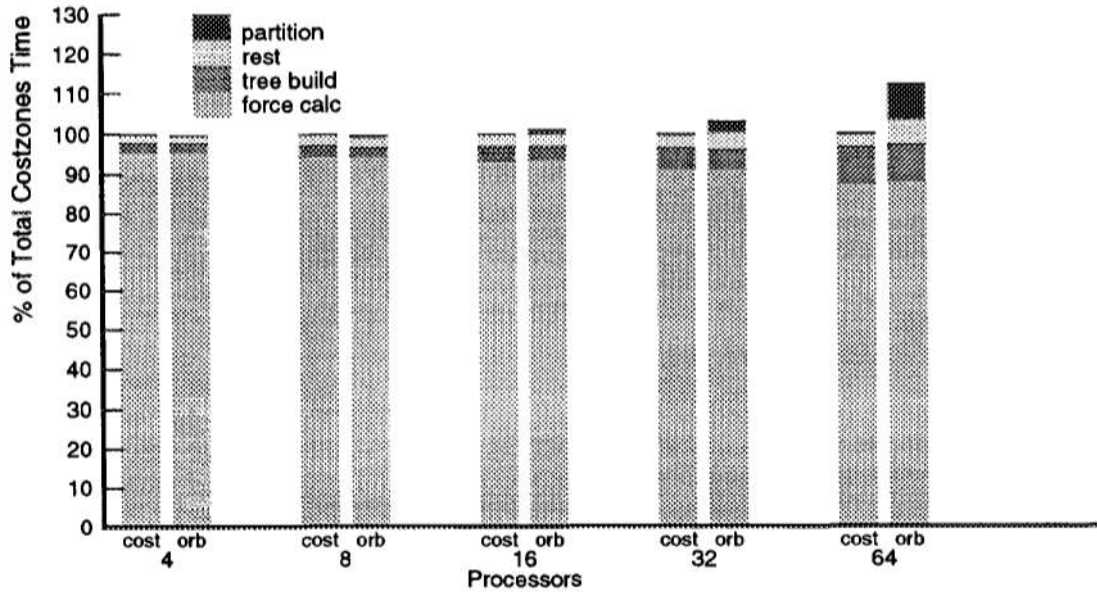


FIG. 14. Barnes-Hut: Execution time profiles for *costzones* and *ORB* on the simulator.

cles introduces structural load imbalances: A processor with k particles in a dense region of the distribution has much more work to do than a processor with k particles in a sparse region. *Compromising load balancing has a more severe performance impact than compromising locality in this application*, at least for reasonable problem sizes running on moderate numbers of processors. The *spacelocality* scheme therefore has the worst overall performance for nonuniform distributions.

The Costzones and ORB Schemes. The *costzones* scheme has low synchronization wait times as well as low communication. It performs best among all schemes in our experiments, and its performance also scales best with the number of processors. (We were able to perform a 16 K particle run on 128 processors of the simulated architecture, in which case the *costzones* scheme yielded a speedup of 118.) This demonstrates that while scalable performance requires dynamic repartitioning for both load balancing and data locality, this can be easily done under programmer control (without resorting to task-stealing for load balancing) by taking advantage of certain insights into the application.

Figure 13b shows that the communication and synchronization overheads incurred by using *costzones* and the more complicated *ORB* are quite similar. However, the speedup curves in Fig. 12 show *costzones* performing significantly better overall. Let us try to understand why. Figure 14 shows profiles of execution time for the simulator runs whose speedups were shown in Fig. 12b. Every bar shows the percentage of time spent in different phases of the application, with the number of processors shown under that bar. The percentages are normalized so that the execution time of the *costzones* scheme with the given number of processors is 100%.

The profiles show that the two schemes are almost equally effective at obtaining their goals in partitioning the force computation phase. However, what makes the overall performance of *ORB* worse is the time spent in the *ORB* partitioning algorithm. Despite our efforts to optimize the implementation of this algorithm,¹⁴ the time spent in it is much larger than that spent in *costzones* partitioning, particularly for larger numbers of processors. The overhead of *costzones* partitioning scales extremely slowly with the number of processors, while that of *ORB* increases much more rapidly. Similar results were observed for other problem sizes on DASH. There is a $\log P$ term in parallel *ORB*, the constant factors are much higher, and the root finder does not parallelize very well for large numbers of processors. Since *costzones* requires processors to descend only a very small number of levels of the tree, its cost is negligible. The simpler *costzones* partitioning mechanism, which can be implemented in a few lines of code, thus yields better performance than the more complex and sophisticated *ORB*, at least for the problem sizes we have studied.¹⁵

¹⁴ Most of the time in *ORB* partitioning is spent in the root-finder that finds the bisector in a given direction. We experimented with different root-finding algorithms for discrete functions, such as the bisection and Van Wijngaarden-Dekker-Brent algorithms described in [4, 9]. The best performance we obtained was from a decisection algorithm (similar to bisection, except that the currently guessed domain is split into 10 equal subdomains rather than 2 at every step in the root-finder). We use the decisection algorithm in the results presented in this paper.

¹⁵ It is of course possible to trade off partitioning frequency with force-calculation efficiency. However, we always find *costzones* to outperform *ORB*. It is also possible to repartition incrementally rather than completely in *ORB*, but this adds a level of complexity and its performance implications are as yet unclear.

7.6.1. The Parallel Tree-Building Bottleneck

Unfortunately, Fig. 14 exposes a potential performance bottleneck when using larger numbers of processors: The fraction of the execution time spent in the tree-building phase increases with the number of processors, which means that the tree-building phase does not speed up quite as well as the other phases. Let us examine how the tree-building algorithm works and see if we can improve its parallel performance.

The sequential tree-building algorithm works as follows. Proceeding one at a time, every particle is loaded into the root of the tree and percolated down to the current leaf cell in which it belongs. If the leaf cell is empty, the particle is simply inserted. Otherwise, the leaf cell is subdivided (perhaps recursively) until there is only one particle per leaf.

The obvious way to parallelize this algorithm is to have processors insert their particles into the shared tree concurrently, using the same basic sequential algorithm and synchronizing as necessary. This is the parallel algorithm for which results were shown in Fig. 14. Every time a processor wants to modify a tree cell (insert a particle in or subdivide a leaf cell), it must first lock that cell for mutual exclusion.¹⁶ As the number of processors increases, the overhead of executing these locks and unlocks becomes substantial and hurts performance.

Contention for the locks is reduced by the physical contiguity of the partitions yielded by *costzones* and *ORB*. These schemes effectively divide up the tree into distinct sections and assign a processor to each section. This means that while there is significant lock contention for the top levels of the tree, contention is very small at lower levels. However, the uniprocessor overhead of executing the lock and unlock routines remains at all levels.

Fortunately, the same physical contiguity of partitions also allows us to construct a better parallel tree-building algorithm, with less locking overhead and less contention at higher levels. This algorithm splits the parallel tree construction into two steps. First, every processor builds its own version of the tree using only its own particles.¹⁷ There is no synchronization in this step. Second, these individual trees are merged together into the single global tree used in the rest of the computation. Details of the merging algorithm can be found in [24]. Since entire subtrees, rather than individual particles, are usually merged into the global tree, the number of times a processor has to lock a cell in the global tree is very small. The reduction in locking necessitates some extra computation, however, since the local trees have to be merged after they are built.

¹⁶ Note that accesses to cells during the traversal to determine the appropriate leaf do not require locking, but a cell that is locked for modification must be tested to ensure that its state is indeed what was observed before locking.

¹⁷ This step is made more efficient by having a processor remember where in its tree it inserted its last particle and start traversing the tree from there for the next particle, rather than from the root.

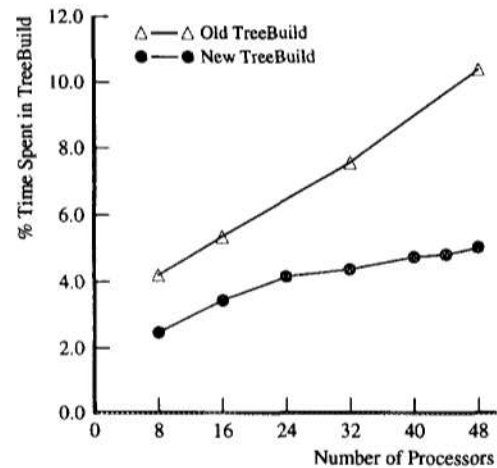


FIG. 15. Barnes-Hut: Percentage of total execution time spent building the tree on DASH.

Figure 15 shows the percentage of the total execution time of the application on the DASH multiprocessor that is spent in building the tree, using the old and new tree building algorithms. The problem size used is the same as the one for which speedups are presented in Fig. 12a. The new algorithm clearly performs better than the old one, particularly as the number of processors increases. We also find that as problem sizes grow for the same number of processors, the fraction of time spent in tree-building decreases. However the fraction increases as the number of processors increases for the same problem size. If the fraction of time spent building the tree becomes large, it might be useful to incrementally modify the tree rather than rebuilding it every time-step. Results show that incremental tree-building can help, but only when the maximum number of particles per leaf cell is allowed to be ten or more.

8. THE FMM APPLICATION

8.1. The Approaches

The basic approaches to partitioning the FMM application are similar to those used in Barnes-Hut. Partitioning in the FMM, however, is complicated by three facts: (i) the basic units of partitioning are cells which, unlike particles, do not persist across time-steps (since both the particle distribution and even the bounding box change), (ii) force-computation work is associated not only with the leaf cells of the tree but with internal cells as well, and (iii) cells undergo different numbers and types of interactions, each involving a different amount of work.

Fact (i) has several implications:

- There is no partitioning scheme analogous to the *static* scheme used in the Barnes-Hut application.
- The work counting needed for load balancing is more complicated, since the work count for a leaf cell has to be

transferred down to its particles (which persist across time-steps) and then back up to the new leaf cells that those particles fall into in the next time-step.¹⁸ It is significantly more expensive and less effective to do these transfers for internal cells. The cost of an internal cell is therefore simply computed from the number of cells in its V list (see subsection 3.1.2), since V list interactions dominate over X list interactions, which are the only other types of interactions that internal cells undergo.

- ORB partitioning is further complicated by having cells be the unit of partitioning, as we shall see.

Fact (ii) indicates that both the *costzones* and *ORB* schemes used in the Barnes–Hut application may have to be extended to partition internal nodes as well. Fact (iii) complicates work counting further, even ignoring the fact that cells do not persist across time-steps. It does not suffice to simply count the number of interactions per cell in the adaptive FMM, or even the number of interactions of different types. We have to compute a cycle count for every cell–cell interaction computed (for example, the interaction of a cell with another cell in its V list). The cycle count for a particular interaction is computed by using a simple function for that type of interaction, parametrized by the number of expansion terms being used and/or the number of particles in the cells under consideration. The cost of a leaf cell is the sum of these counts in all interactions that the cell computes. As mentioned above, the costs of internal cells are computed from only the V list interactions. As in Barnes–Hut, the work counting is done in parallel as part of the computation of cell interactions, and its cost is negligible. Let us examine the analogs of four of the five partitioning schemes used in the Barnes–Hut application (since the *static* scheme does not make sense here, see above).

The *loadbalance* scheme requires no structural modification for use in the FMM application, other than using cells rather than particles as the units of parallelism. The *costzones* scheme as described for Barnes–Hut in subsection 7.3.2 considers only the costs of leaves of the tree when partitioning. This is all that is required for the Barnes–Hut method, since internal cells have no work associated with them in the force calculation phase. Our initial implementation of *costzones* for the FMM, called *costzones-initial*, is a direct translation of that scheme. Internal cells are assigned in the same way as they were for locality in the Barnes–Hut case: to the processor that owns the majority of their children.

As mentioned earlier, internal cells also have substantial costs in the FMM, which should be considered in load balancing. We therefore extend the *costzones* scheme to partition internal cells based on their costs as well. We call

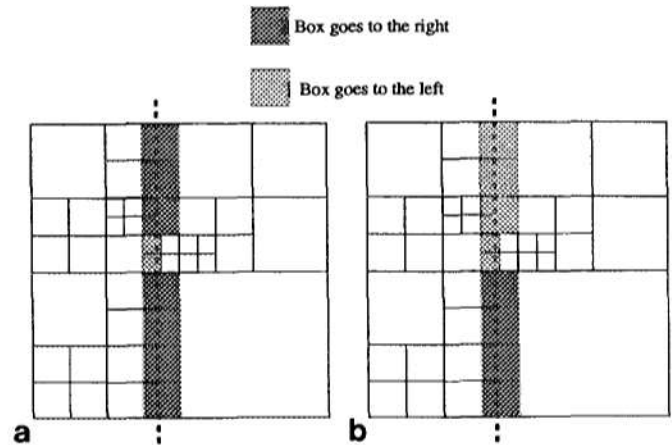


FIG. 16. Partitioning of border cells in ORB for the FMM. (a) *ORB-initial*; (b) *ORB-final*.

this scheme *costzones-final*. In this scheme, an internal cell holds not just the sum of the costs of all leaves within it (as in *costzones-initial*), but the sum of the costs of all cells (leaf or internal) within it plus its own cost. In addition, it holds its own cost separately as well. In the traversal of the planarized tree that performs *costzones* partitioning, a processor examines cells for potential inclusion in its partition in the following order: the first two children (from left to right), the parent, and the next two children.¹⁹ Pseudocode for the *costzones-final* scheme can be found in [24].

The fact that the unit of parallelism is a cell rather than a particle complicates ORB partitioning in the FMM. When a space is bisected in ORB, several cells are likely to straddle the bisecting line (unlike particles, see Fig. 16). Our first implementation, called *ORB-initial*, directly parallels the *ORB* scheme used in Barnes–Hut. Cells, both leaf and internal, are modeled as points at their centers for the purpose of partitioning, just as particles were in Barnes–Hut. At every bisection in the ORB partitioning process, therefore, a cell that straddles the bisecting line (called a *border cell*) is given to whichever subspace its center happens to be in.

As we shall see, this treatment of border cells leads to significant load imbalances. Since cells are always split in exactly the same way (into four children of equal size), the centers of many cells are likely to align exactly with one another in the dimension being bisected. These cells are in effect treated as an indivisible unit when finding a bisector. If a set of these cells straddles a bisector, as is very likely, the entire set will be given to one or the other side of the bisector (see Fig. 16a), potentially giving one side a lot more work than the other.²⁰ The resulting load imbalance may be compounded in successive bisections.

¹⁸ We do this in the following way. The profiled cost of a leaf cell is divided equally among its particles. In the next time-step, a leaf cell examines the costs of its particles, finds the cost value that occurs most often, and multiplies this value by the number of particles to determine its cost.

¹⁹ Recall that we use a two-dimensional FMM, so that every cell has at most four children.

²⁰ This situation is even more likely with a uniform distribution, where many cells will have their centers exactly aligned in the dimension along which a bisection is to be made.

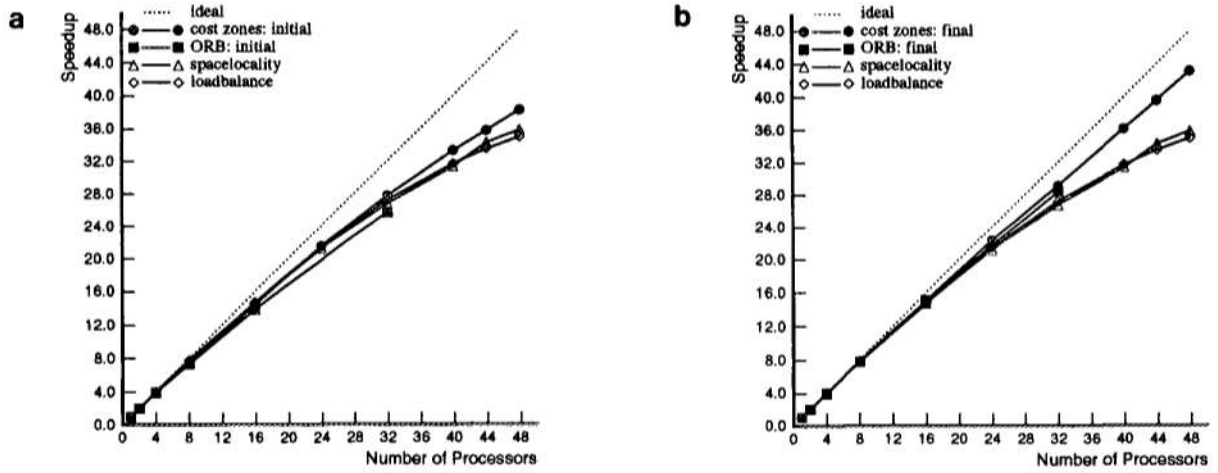


FIG. 17. Speedups for the FMM application on DASH ($n = 32\text{ K}$, $\varepsilon = 10^{-10}$). (a) Initial schemes; (b) final schemes.

To solve this problem, we extend the ORB method as follows. Once a bisector is determined (by *ORB-initial* above), the border cells straddling the bisector are identified and repartitioned. The repartitioning of border cells preserves the contiguity of partitions and equalizes costs as far as possible (see Figure 16b). A target cost for each subdomain is first calculated as half the total cost of the cells in both subdomains. The costs of the border cells are then subtracted from the costs of the subdomains that *ORB-initial* assigned them to in that bisection. Next, the border cells are visited in an order sorted by position along the bisecting axis and assigned to one side of the bisector until that side reaches the target cost. The rest of the border cells are assigned to the other side of the bisector. We call this scheme that repartitions border boxes *ORB-final*. Note that whether or not border cells are repartitioned, the partitions yielded by *ORB* have lost the geometrically regular structure that the *ORB* partitions in the Barnes–Hut application had (see Fig. 16).

The *spacelocality* scheme is also very similar to the *spacelocality* scheme in the Barnes–Hut application: Cost-zones-style partitioning is used (see below), but every cell is assumed to have the same cost.

8.2. Results

A similar two-cluster input distribution is used in the FMM application as in the Barnes–Hut, except that it is in two dimensions rather than three. Measurements on the DASH multiprocessor are made with 32 K particles and an accuracy²¹ of 10^{-10} . Measurements with other reasonable numbers of particles and accuracies show similar trends across partitioning schemes. On the simulator, we were

not able to run problems with more than 8 K particles and a precision of 10^{-6} in a reasonable amount of time. With the 40 particles per leaf cell that we use on DASH and that Greengard recommends (see subsection 3.1.2 and [14]), this leads to a very small number of cells and hence to large load imbalances. We therefore allow a maximum of 5 particles per leaf cell on the simulator, to obtain more cells and hence more concurrency. Since this still causes significant load imbalances with the larger numbers of processors, we do not present speedups obtained with the simulator but use them only to compare the load balancing and communication behaviors of different schemes. As in the Barnes–Hut application, all measurements are started at the end of the second time-step.

Figure 17a compares the performance on DASH of the four schemes that are direct analogs of the schemes used in the Barnes–Hut case: *loadbalance*, *spacelocality*, *costzones-initial*, and *ORB-initial*. The *costzones-initial* scheme clearly performs better than the others, although even its performance falls off as more processors are used owing to its naive treatment of internal cells of the trees (which represent over 25% of the total force computation cost for this particle distribution). The *ORB-initial* scheme performs relatively poorly, owing to the problem with border cells which account for a significant portion of the cost at each bisection.

Figure 17b shows the speedups obtained when *costzones-initial* and *ORB-initial* are replaced by *costzones-final* and *ORB-final*. The best performing scheme in all cases is *costzones-final*. The improvement in partitioning parent cells clearly avoids the deterioration of speedup observed with *costzones-initial*. The *ORB-final* scheme yields substantial improvement over *ORB-initial*, which makes it almost as good as *costzones-final* for up to 32 processors with this nonuniform distribution.

Figure 18 shows that *ORB-final* and *costzones-final* have similar communication overhead, while *ORB-final* has slightly worse load balancing behavior. The net result is

²¹ Recall from subsection 3.1.2 that the force calculation accuracy governs the number of terms used in the multipole and local expansions. The number of terms is equal to $\log_c \varepsilon$, where ε is the accuracy and $c = 1.828$.

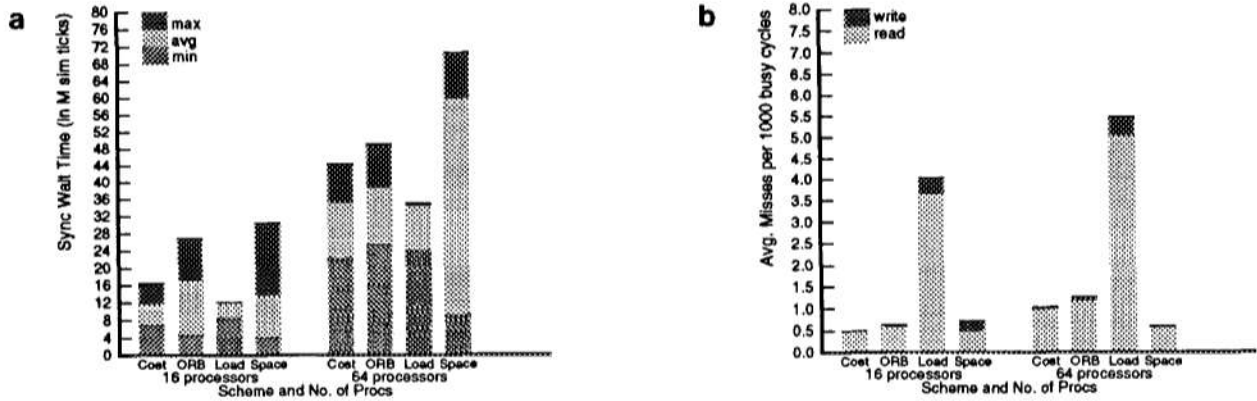


FIG. 18. FMM behavioral statistics on the simulator ($n = 4K$, $\epsilon = 10^{-4}$). (a) Synchronization wait times; (b) communication.

similar to that for Barnes–Hut: the force computation phase performs slightly better with *costzones-final*, and the higher partitioning cost in *ORB-final* causes the overall application performance of *costzones-final* to be increasingly better than *ORB-final* as the number of processors increases.

Thus, the FMM application also requires dynamic repartitioning for both load balancing and locality. However, because the units of partitioning are cells—each with many particles—and because internal nodes of the tree have significant amounts of work associated with them, direct translations of the *costzones* and *ORB* schemes as used in the Barnes–Hut case do not suffice. Significant enhancements to these schemes are required, as are more complicated work counting and prediction mechanisms than in Barnes–Hut. Finally, the simplicity of *costzones* partitioning versus *ORB* is further emphasized by the FMM, since the partitioning complexity that the FMM adds for effective performance is greater in *ORB* than in *costzones*: Both have to consider the costs of internal cells, while only *ORB* has also to repartition border cells at every bisection.

By using the appropriate partitioning techniques, we find that both the prominent hierarchical methods for classical N -body problems can be made to yield excellent and seemingly scalable parallel performance. There is no need to resort to dynamic task-stealing, primarily because the physical systems that classical problems model evolve slowly with time, and because low-overhead work profiling metrics can be identified in both algorithms. The approaches to partitioning that are effective in both the Barnes–Hut and the FMM applications are quite similar. The two approaches also often require similar traversals of and operations on tree data structures. This suggests that libraries of the partitioning/manipulation techniques (which can be customized to a particular application by passing them the appropriate user-defined primitive functions) might prove very useful to parallel programmers. In the next section, we will see how the very different characteristics of the radiosity problem—from those of the classical problems—require very different kinds of techniques to obtain good speedups on the hierarchical radiosity method.

9. THE RADIOSITY APPLICATION

9.1. The Straightforward Approaches

The main parallel loop in every iteration is a loop over all polygon–polygon interactions. Our first two approaches exploit this parallelism in much the same way as an automatic scheduler might, to produce the following partitioning schemes which are analogous to the *static* and *load-balance* schemes, respectively, in Barnes–Hut:

- *Static*: Every processor is assigned a set of input polygons and their interactions, so that there are an equal number of initial polygon–polygon interactions per processor. The assignment is the same in every iteration. A processor works on only the interactions it is assigned and the patch–patch interactions that they spawn.
- *Single-queue-polygon (SQ-Pol)*: Process dynamically obtain initial polygon–polygon interactions until there are none left. Once a processor obtains such an interaction, it performs all the computation for it and its children.

Speedups for these schemes on the DASH multiprocessor (as well for several other schemes which we will describe) are shown in Fig. 19. The input is a room scene used in the paper that presented the sequential algorithm [16]. The speedup with the *static* scheme saturates at about threefold and with the *single-queue-polygon* scheme at about eightfold. In the case of *single-queue-polygon*, the reason for the saturation is that the granularity of a task (a polygon–polygon interaction) is very large compared to the number of tasks. For the same reason, load balancing in the *static* scheme is not helped by the averaging effect of large partitions, as it was in Barnes–Hut, particularly since the work per input polygon varies widely across polygons (depending on their radiosities, areas, and relationships to other polygons). The naive schemes thus perform poorly in this application.

9.2. Techniques for Load Balancing and Data Locality

Let us now see what it takes to incorporate data locality and load balancing in this application. We begin with a discussion of locality and then examine load balancing.

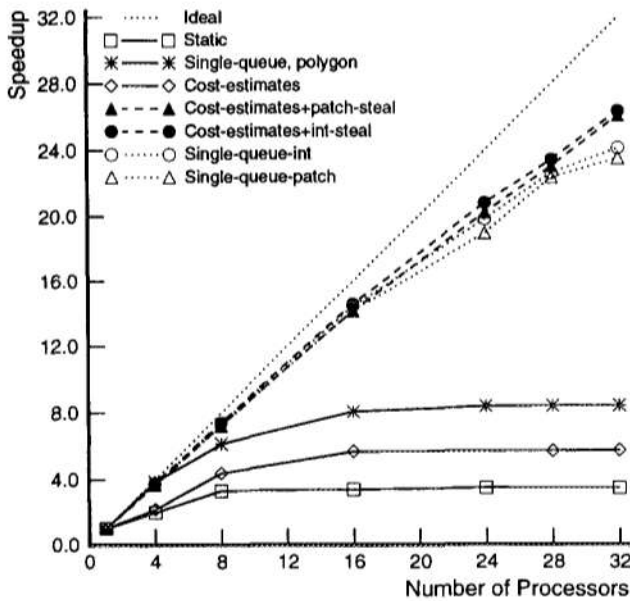


FIG. 19. Speedups for the radiosity application on DASH.

9.2.1. Data Locality

One important difference in the radiosity application from the classical N -body problems is that physical locality in the problem domain is not nearly as important. Since the distance between two patches is not the only criterion governing the degree of interaction—the angle and visibility between the patches are also very important, as is the radiosity of each patch—partitioning techniques analogous to *costzones*, *ORB*, and *space-locality* do not apply.

There are three primary forms of locality in this application. First, a form of object locality can be obtained by having a processor work on interactions involving the same input polygons and their subpatches in every iteration. Second, locality can be exploited across patch computations by processing patches in an appropriate order. Patches are refined depending on the errors computed in their interactions. Because sibling patches in a quadtree are physically adjacent and belong to the same polygon, interactions from these siblings to another patch (in another polygon) usually have similar amounts of error. Thus, if an interaction from one of the siblings causes the destination patch to be subdivided, interactions from the other siblings to the same patch are likely to cause subdivision as well. As a result, the sets of patches with which sibling nodes interact overlap substantially. For the input scenes we used, we found that of all the interactions at a given level of subdivision, 81% are made to the same patch from all four siblings, and 92% are made to the same patch from at least three of four siblings. This high coherence of the interactions can be exploited for locality by performing a breadth first traversal of the quadtree, so that sibling patches are processed consecutively.

The third form of locality can be exploited in visibility testing. Visibility testing involves shooting a finite number

of rays between random points on the two interacting patches and counting the number of these rays that are intercepted by other intervening patches. The visibility between the two interacting patches is the fraction of rays that are not intercepted. The BSP tree is used to find intercepting patches efficiently. Basically, a visibility calculation between two patches involves traversing a pruned inorder path in the BSP tree between the polygons that those patches belong to [11]. Locality can therefore be exploited by trying to ensure that the same subset of BSP-tree nodes is traversed by a processor in successive visibility calculations. It can be shown that this locality is easily obtained by traversing the quadtree representing an input polygon depth-first rather than breadth-first [24].

Thus, while breadth-first traversal of the quadtrees is preferable for locality in interaction refinement, depth-first is preferable for locality in visibility testing. Since visibility testing and interaction refinement are performed during the same traversal of a quadtree, we have to choose one of the traversal schemes. We select depth-first traversal, because visibility testing consumes more than 90% of the sequential running time and because most of the cache misses occur in visibility testing. Fortunately, depth-first traversal preserves a significant amount of locality in interaction refinement as well. For example, leaf-level siblings are visited successively even under depth-first search, and these nodes are a large fraction (about three-quarters) of the nodes in the system.

The *static* scheme we described earlier both exploits object locality across iterations and provides depth-first traversal of the quadtrees. It therefore satisfies our locality criteria, and there is not much more a programmer can or needs to do about locality. The more important issue in this application is load balancing, and we examine that next.

9.2.2. Load Balancing

Since we would like to obtain load balancing without losing control over data locality if possible, and since the *static* scheme provides excellent data locality, we try to incorporate load balancing in the *static* scheme. Our first attempt was to use the same approach that was successful in the classical applications: using work profiles from one iteration to predict those in the next. This approach was unsuccessful for two reasons:

- There are only a few (less than 10) iterations in this application—unlike the hundreds of time-steps in the gravitational case—and a disproportionately large fraction of the work is done in the first iteration. No profiled information is available for this iteration.
- Unlike classical N -body problems, the domain being simulated in radiosity is not a physical system evolving slowly with time. Thus, the work associated with a patch in one iteration²² is *not* a good indicator of the work associ-

²² Note that iterations in this application are a mathematical construct used to obtain convergence and have nothing with physical time, unlike the time-steps in the classical applications.

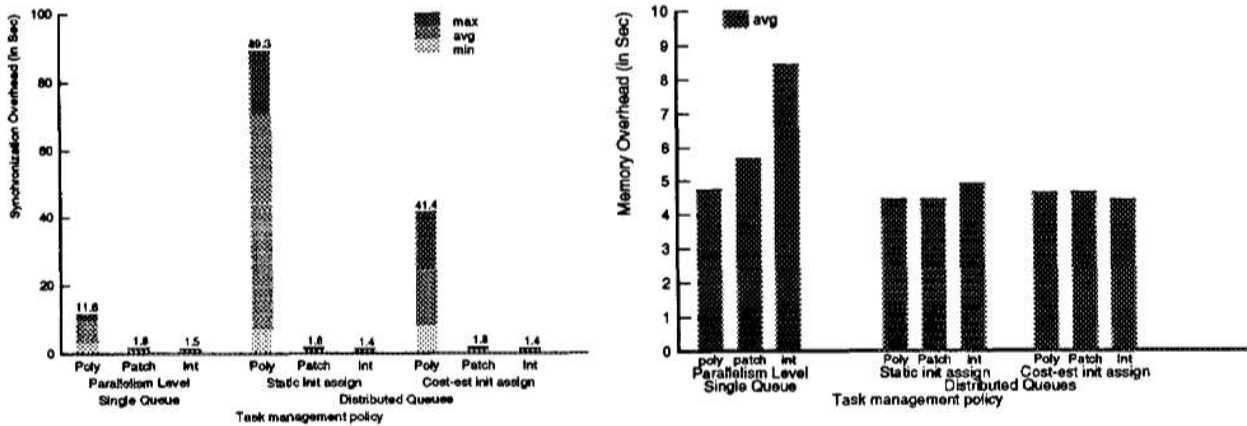


FIG. 20. Synchronization and memory overhead for the radiosity application on DASH ($p = 16$). Execution time for best case (rightmost bar) is 22 s.

ated with it in the next. A patch has significant work associated with it in only those iterations in which many of its interactions lead to subdivisions. These subdivisions happen unpredictably, depending on when current radiosity levels and form factors cause the BF_{err} value for an interaction to cross a certain threshold.

We experimented with several cost-estimating and profiling functions. Since there is a decreasing trend in the cost of a patch across iterations, we even tried to use a history of profiled costs rather than the profiled cost in a single iteration. The best scheme that we could come up with is represented as *cost-estimates* in Fig. 19. In it, we give exponentially decaying weights (1.0, 0.5, 0.25) to the costs in the three previous iterations. While the performance of this scheme is better than that of the *static* scheme, it does not nearly solve the load balancing problem.

Our only resort for incorporating load balancing is through dynamic task-stealing. However, we must use finer-grained tasks than were used in the *single-queue-polygon* scheme. To reduce contention and preserve locality, we use a distributed task-queueing mechanism in which every processor has its own task queue. In every iteration of the radiosity algorithm, we first used one of the mechanisms discussed above—*static* or *cost-estimates*—to provide every task queue with an initial set of polygon-polygon interactions. If a processor subdivides a patch in the course of computing interactions, it inserts a task for each created patch into its own queue. A task for a patch includes all the interactions that task must perform. It is created when all the initial interactions for the patch are determined, i.e., when its parent patch has been completely processed.

When a processor finds no more work in its task queue, it steals tasks from other processors' queues. The following mechanisms are used to preserve locality in the task-queue system. A processor inserts tasks that it creates at the head of its queue. It always dequeues tasks from the head of its own queue (which yields the desired depth-first search of

the quadtrees). However, when it steals from another queue, it dequeues the task at the tail of that queue. This maximizes the likelihood of stealing a large task and exploiting the locality within it and the tasks it engenders.

The scheme that incorporates patch stealing is called *cost-estimates + patch-steal* ($C + PS$).²³ Figure 19 shows that the incorporation of patch stealing makes a dramatic difference to parallel performance and provides very good speedups. Measurements obtained with the MTOOL performance debugger [12] on DASH (Fig. 20) show that the techniques used to maintain locality while stealing cause the stealing to contribute no appreciable increase in memory referencing overhead for these numbers of processors (16, in the figures). Figure 20 shows the overheads due to time spent in the memory system and time spent waiting at synchronization events (such as the final barrier at the end of the radiosity computation in an iteration).

The granularity of tasks so far has been a patch and all its interactions. Since we find that there is still some small load imbalance at the end of an iteration, we examine the impact of using a smaller task granularity, that is, the finest level of parallelism described in subsection 3.2.2. A task now is a single patch-patch interaction, so that the time-consuming visibility computation of a patch with different patches on its interaction list can be done in parallel. If an interaction subdivides one of the patches and thus spawns new interactions, then each of the newly created interactions is a new task which is placed on the creating processor's task queue. This scheme is called *cost-estimates + int-steal* ($C + IS$). The performance debugger indicates that this scheme reduces load imbalance but increases memory system overhead. The net result is only a small improvement in speedup for this problem and number of processors.

Finally, to examine the impact of ignoring locality completely, we implemented schemes in which the patch and interaction tasks of $C + PS$ and $C + IS$ are inserted in and dequeued from the head of a single queue. These

²³ *Static + patch-steal* shows similar results.

schemes are called *single-queue-patch* (*SQ-P*) and *single-queue-int* (*SQ-I*) in Fig. 19. Their performance is not too much worse than that of $C + PS$ and $C + IS$, since there is enough locality even within a patch-patch interaction for a machine of this size, and since contention for the single queue is also small compared to task granularity. This situation, however, is likely to change for larger machines.

We also examined the performance of the different schemes with a room scene twice as large and found the results to be quite similar (slightly larger overall speedups and slightly smaller differences between the schemes that are load balanced). At least for moderately sized parallel machines, then, load balancing is a much more important performance issue than locality for this application, just as it was for the classical applications.

10. SUMMARY AND CONCLUDING REMARKS

We studied the process of obtaining effective speedups through parallelism on the major hierarchical N -body methods that have been proposed so far. Our study included the two most powerful methods for classical N -body problems—the Barnes–Hut algorithm and the fast multipole method—using a galactic simulation as our example problem. In addition, we examined a hierarchical method for radiosity calculations in computer graphics, an application that uses the same basic approach as the other two but that has some very different characteristics. The focus of our study was on partitioning/scheduling techniques that simultaneously obtain load balancing and exploit data locality.

We found that all the applications can be made to yield excellent and scalable parallel performance. However, static or straightforward dynamic parallel implementations of the sort that a compiler or automatic scheduler might produce²⁴ do not scale. This is because the workload distribution and communication characteristics in the applications are nonuniform and change as the computation proceeds. The characteristics of classical N -body problems allow partitioning techniques based on a similar approach to be successful for the Barnes–Hut and fast multipole methods that are used to solve them, even though the techniques that are successful in the Barnes–Hut method have to be extended significantly for the FMM. Since several phases in these methods require similar operations to be performed on tree data structures, libraries of tree manipulation and tree/space partitioning techniques (which can accommodate different user-defined primitive functions) might be very helpful to a parallel programmer for classical N -body problems.

Although the radiosity application uses an algorithm based on hierarchical N -body methods, its characteristics

are very different from those of the classical applications. As a result, static partitioning and scheduling techniques perform much more poorly in the radiosity application. Even the more sophisticated mechanisms, such as *cost-zones* and *ORB*, that were successful in providing load balancing and locality in the classical applications do not apply to the radiosity problem. Thus, although the partitioning techniques that succeed on the classical N -body applications are similar, no single technique can be claimed to be successful for applications of the hierarchical N -body approach.

Despite our best efforts, we could not find an effective predictive mechanism that could provide load balancing without on-the-fly task-stealing in the radiosity application. Fortunately, however, there is enough locality within tasks that relatively simple stealing mechanisms do not compromise locality too much. The result is that the hierarchical radiosity application also yields good parallel performance that is likely to scale to large numbers of processors.

In other work [24, 28], we show that the nonuniform, dynamically changing nature of problems that hierarchical N -body methods are applied to, together with their need for long-range communication in the physical domain, cause multiprocessors that support a shared-address-space communication abstraction to have substantial advantages in design and programming complexity over machines that support only explicit message passing among private address spaces. We also argue that this message-passing complexity translates directly to substantial performance overheads, which can be avoided with a shared address space. The key issue that we have not yet actually demonstrated is whether our shared-address-space implementation will scale to much larger numbers of processors, although extrapolations from the characteristics we have observed indicate that it should [28]. We are currently investigating this question.

ACKNOWLEDGMENTS

We thank Joshua Barnes for providing us with the sequential Barnes–Hut program, Dennis Roger, Eric Bruni, and Maneesh Agrawala for their help with implementation, and John Salmon, Rohit Chandra, and Kourosh Gharachorloo for various discussions. This work was supported by DARPA under Contract N00039-91-C-0138.

REFERENCES

1. Aarseth, S. J., Henon, M., and Wielen, R. *Astronom. Astrophys.* **37** (1974), 75–81.
2. Appel, A. A. An efficient program for many body simulation. *SIAM J. Sci. Statist. Comput.*, **6** (1985), 85–93.
3. Barnes, J. E., Hut, H. A hierarchical $O(N \log N)$ force calculation algorithm. *Nature* **324**, 4 (1986), 446–449.
4. Brent, R. P. *Algorithms for Minimization without Derivatives*. Prentice-Hall, New York, 1973.
5. Chan, T. Hierarchical algorithms and architectures for parallel scientific computing. In *Proceedings of ACM International Conference on Supercomputing*, May 1990.

²⁴ If it knew the appropriate parallelism, which we believe would be impossible for today's compiler technology to find.

6. Chorin, A. J. Numerical study of slightly viscous flow. *J. Fluid Mech.* **57** (1973), 785–796.
7. Chua, K., Leonard, A., Pepin, F., and Winckelmans, G. Robust vortex methods for three-dimensional incompressible flows. In *Proceedings of Symposium on Recent Advances in Computational Fluid Dynamics*, 1988.
8. Cohen, M. F., and Greenberg, D. P. The hemi-cube: A radiosity solution for complex environments. In *Proceedings of SIGGRAPH*, 1985.
9. Press, W., et al. *Numerical Recipes in C*. Cambridge Univ. Press, Cambridge, UK, 1988.
10. Fox, G. C. A graphical approach to load balancing and sparse matrix vector multiplication on the hypercube. In *Numerical Algorithms for Modern Parallel Computer Architectures*, Springer-Verlag, Berlin/New York, 1988, pp. 37–62.
11. Fuchs, H., Abram, G. D., and Grant, E. D. Near real-time shaded display of rigid objects. In *Proceedings of SIGGRAPH*, 1983.
12. Goldberg, A. J., and Hennessy, J. L. Performance debugging shared memory multiprocessor programs with MTOOL. In *Proceedings of Supercomputing '91*, Nov. 1991, pp. 481–490.
13. Goldschmidt, S. R., and Davis, H. Tango introduction and tutorial. Tech. Rep. CSL-TR-90-410, Stanford University, 1990.
14. Greengard, L. *The Rapid Evaluation of Potential Fields in Particle Systems*. Assoc. Comput. Mach., New York, 1987.
15. Greengard, L., and Rokhlin, V. A fast algorithm for particle simulations. *J. Comput. Phys.* **73** (1987), 325–348.
16. Hanrahan, P., Salzman, D., and Aupperle, L. A rapid hierarchical radiosity algorithm. In *Proceedings of SIGGRAPH*, 1991.
17. Hernquist, L. Hierarchical N-body methods. *Comput. Phys. Comm.* **48** (1988), 107–115.
18. Jernigan, J. G., and Porter, D. H. A tree code with logarithmic reduction of force terms, hierarchical regularization of all variables and explicit accuracy controls. *Astrophys. J. Suppl.* (1989), 871.
19. Katzenelson, J. Computational structure of the N-body problem. *SIAM J. Sci. Statist. Comput.* **10**, 4 (1989), 787–815.
20. Lenoski, D., et al. The directory-based cache coherence protocol for the DASH multiprocessor. In *Proceedings of the 17th Annual International Symposium on Computer Architecture*, May 1990, pp. 148–159.
21. Rokhlin, V. Rapid solution of integral equations of classical potential theory. *J. Comput. Phys.* **60** (1985), 187–207.
22. Rothberg, E., Pal Singh, J., and Gupta, A. Working sets, cache sizes and node granularity issues for large-scale multiprocessors. In *Proceedings of the 20th Annual International Symposium on Computer Architecture*, May 1993.
23. Salmon, J. K. Parallel hierarchical N-body methods. Ph.D. thesis, California Institute of Technology, Dec. 1990.
24. Pal Singh, J. Parallel hierarchical N-body methods and their implications for multiprocessors. Ph.D. thesis, Stanford University, Feb. 1993.
25. Pal Singh, J., and Hennessy, J. L. Data locality and memory system performance in the parallel simulation of ocean eddy currents. In *High Performance Computing II*. North-Holland, Amsterdam, 1991, pp. 43–58; Tech. Rep. CSL-TR-91-490, Stanford University.
26. Pal Singh, J., and Hennessy, J. L. Data locality and memory system performance in the parallel simulation of ocean eddy currents. In *High Performance Computing II*. North-Holland, Amsterdam, 1991, pp. 43–58; also Tech. Rep. CSL-TR-91-490, Stanford University.
27. Pal Singh, J., Hennessy, J. L., and Gupta, A. Scaling parallel programs for multiprocessors: Methodology and examples. *IEEE Comput.* **26**, 7 (July 1993), 42–50.
28. Pal Singh, J., Hennessy, J. L., and Gupta, A. Implications of hierarchical N-body methods for multiprocessor architecture. Tech. Rep. CSL-TR-92-506, Stanford University, 1992. *ACM Trans. Comput. Syst.*, (May 1995), to appear.
29. Zhao, F. An $O(n)$ algorithm for three-dimensional N-body simulations. Tech. Rep. 995, MIT Artificial Intelligence Laboratory, 1987.

Received December 10, 1991; revised January 20, 1994; accepted September 7, 1994